`transactionStepObserver · from the code, 2026-11-12 (abfff24)`

EMV flows in EasyLink

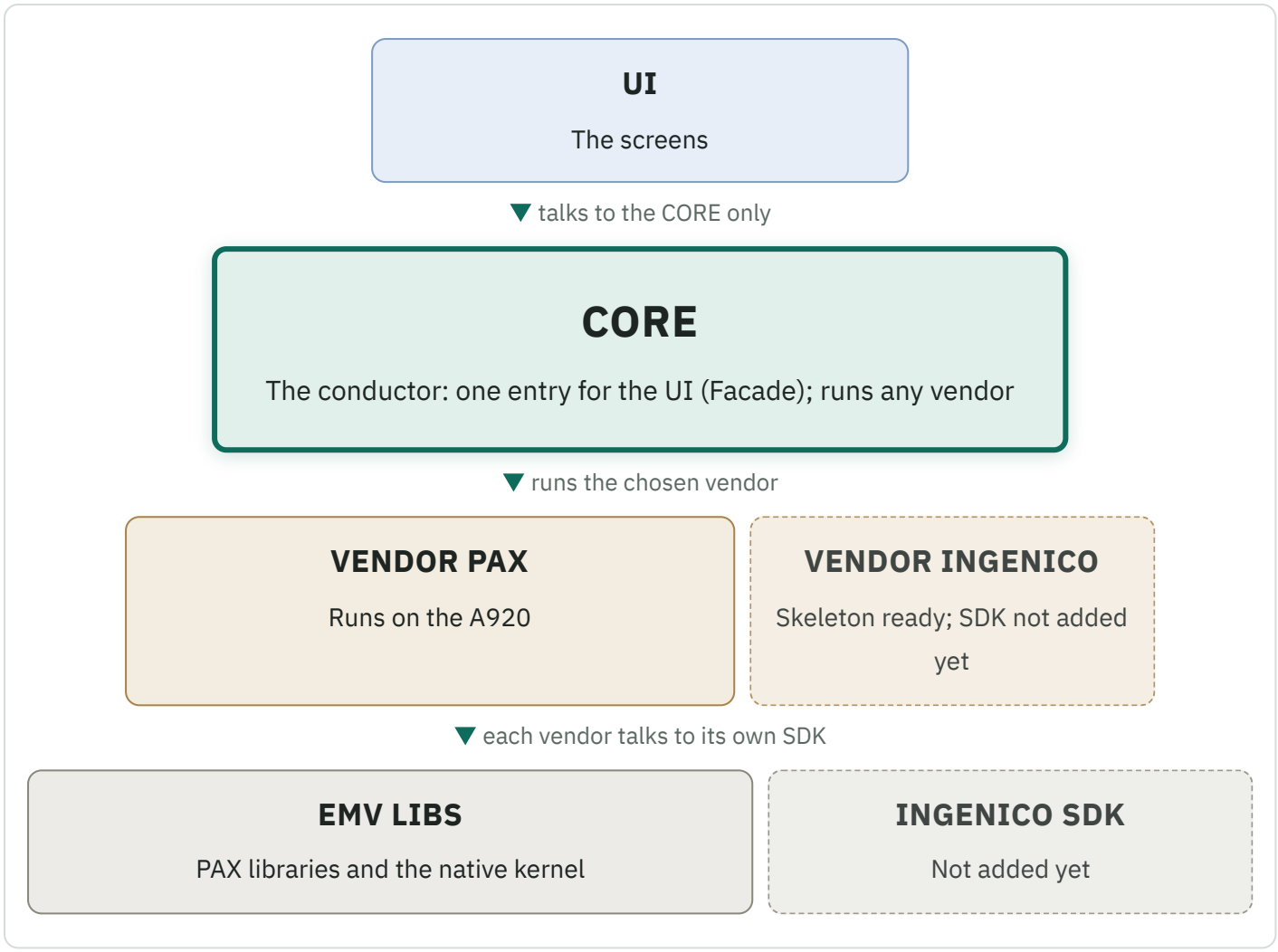
A sale from the Sale button to its end: the card, the kernel, the bank, then the acknowledgement or reversal at the bank. Every name on this page exists in the code, and every arrow is a real call. Everything here describes the branch `transactionStepObserver`.

---

1. Components: who owns what
2. A sale from start to end
3. EMV steps: goToStep and onXxx
4. When the kernel waits: ask
5. The way to the bank
6. The contact path
7. The outcome and the safe end
8. The contactless path
9. EMV params
10. Design principles
11. Using us as an SDK

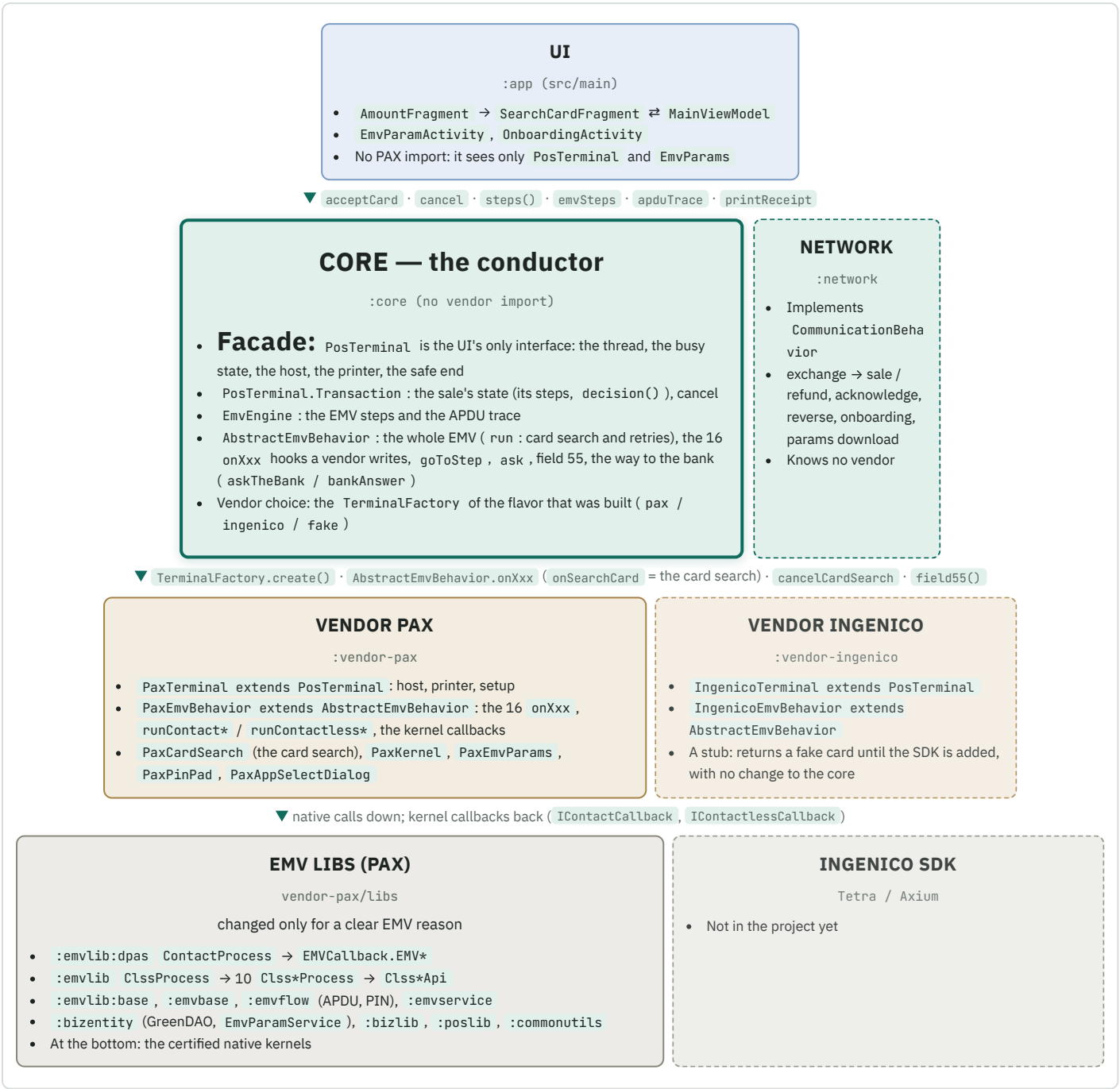
1. Components: who owns what

Each responsibility has one owner. The screen does not know PAX; the EMV knows neither the bank nor the threads; `PosTerminal` is the only door the screen uses.


UI
The screens
talks to the CORE only
CORE
The conductor: one entry for the UI (Facade); runs any vendor
runs the chosen vendor
VENDOR PAX
Runs on the A920
VENDOR INGENICO
Skeleton ready; SDK not added yet
each vendor talks to its own SDK
EMV LIBS
PAX libraries and the native kernel
INGENICO SDK
Not added yet

The layers in detail

Each box is a module, with its main classes. What sits between two layers is the only interface from one to the one below.



- **Calls go down only.** What comes up is the steps (`Transaction.steps()` , `emvSteps`) and the kernel callbacks back to their own vendor.
- **A new vendor** is a new module with a `TerminalFactory` , a `PosTerminal` subclass (setup) and an `AbstractEmvBehavior` subclass (the steps, the card search included). The UI and the CORE do not change (`doc/VENDOR.md`).
- `:vendor-fake` exists for tests and proves the CORE runs with no SDK.

Each layer's parts

One card per layer of the pyramid: its packages, and each class with its job in one line.

UI `:app (src/main)`

The screens. They ask the CORE and show what comes back; they hold no sale state and know no vendor.

Start-up

`EmvEnhanceApp`

Makes one `PosTerminal` for the app's whole life (`TerminalFactory.create`) and sets the host address

The sale

`MainActivity`

The main screen: buttons only

`AmountActivity / AmountFragment`

The sale type and amount, then the card screen

`SearchCardFragment`

The sale screen: the banner, cancel, the result

`MainViewModel`

Binds `PosTerminal` to LiveData: `acceptCard` , `cancel` , the steps (EMV steps per sale), the trace

`Event`

A value read once (survives rotation)

Settings

`EmvParamActivity / EmvParamListFragment`

EMV params: from the cache or the database; the download

`OnboardingActivity`

Registers the terminal with the host; mToken; PIN bypass

`CountryListActivity`

Country and currency

CORE — the conductor `:core`

The sale from start to end, with the same code for any vendor: the UI talks to it alone, and it talks to the vendor through interfaces and abstract classes.

`terminal` — the machine

`PosTerminal`

The Facade, the UI's only entry: one thread, the busy state, the call to the bank (`sendToBank`), the printer

`PosTerminal.Transaction`

One sale: its state (from its steps, `decision()`), its steps (run by `TransactionStepObserver`), cancel, the requests sent to the bank and their safe end

`EmvBehavior`

The interface: `run` · `cancelCardSearch` · `dispose`

`BankRequest`

The EMV's way to the bank: `send(config)` ; `PosTerminal` hands it over with `attach(this::sendToBank)`

`AbstractEmvBehavior`

The shared EMV: `run` (attempts, search, retry: `requestRetry` / `retryCount()`), `goToStep` , `ask` , field 55, the 16 `onXxx` hooks. It lives here because it calls package-private `Transaction` methods

`TerminalBusyException`

A sale started while another is running

`transaction` — the sale

`TransactionConfig`

Type, amount, entry method, field 55, PIN block, `asyncRequestId`

`TransactionType`

sale / refund / cash in

`TransactionStep / Event`

The sale's steps: start, waiting for the card, the bank, the decision, acknowledge / reverse, the end. The steps that drive the sale each have a case

`emv` — the EMV

`EmvEngine`

Sends each EMV step to its runner and the banner, and keeps the APDU trace. No sale state, no retry, no bank

`EmvStep / Event`

The kernel's 17 steps (EMV Book 3)

`EmvTransactionResult`

What was read from the card and the host needs

`Field55`

Builds field 55 from the tags, the same for every vendor

`ApduTrace`

The APDU and step log (debug builds)

`card` — the card

`CardPresence / CardSearchListener`

The card found (`isEmv()` / `isNonEmv()`) and the search events

`EntryMethod`

chip / tap / swipe / manual

`host` — outside interfaces

`CommunicationBehavior`

The host: `connectToHost` · `acknowledge` · `reverse`

`AuthResult`

The host's answer: approved / declined / failed / notSent / pinRequired

`PrinterBehavior`

`cardholder` — waiting for the cardholder

`AbstractAppSelectDialog`

Application choice: the wait and the timeout; never picks the first one by itself

`PinPadBehavior / AbstractPinPadBehavior / PinKey`

The PIN screen: dots and buttons only; encryption is

`param` · `util`

`EmvParams`

The EMV params screen's interface

`EmvLog`

Logging

The printer

`HostDefaults`

A demo host and printer

the SDK's job

NETWORK `:network`

Talks to our host. Implements `CommunicationBehavior` and knows no vendor.

The sale

`SaleCommunicationBehavior`

exchange → sale / refund, acknowledge and reverse; decides for each failure whether the request may have reached the host

`HostApiConnection / HostApiClient`

The endpoints (Retrofit) and the client

`HostHeaders / HostSettings / HostAppKeys`

The headers: the token, the mToken (editable on the device), the fixed keys

`PosEntryModeCodes`

051 / 071 / 901 / 011

`crypto · tlv`

`RsaPublicKeyEncryptor`

Encrypts the sale key (TEK) with the host's key

`AesEnvelopeCrypto`

Encrypts the request and decrypts the answer

`BerTlv / IssuerResponseFields`

Read the ARPC and scripts from the host's answer

`onboarding · params`

`OnboardingClient / OnboardingState`

Terminal registration, keys, the token and its refresh

`EmvParamDownloadClient`

Downloads the params file (bytes only; PAX parses it)

`EnvironmentProvider`

The build's host address

`PinBypassSettings`

Whether PIN bypass is allowed

VENDOR PAX `:vendor-pax`

PAX A920: implements the CORE's interfaces with the PAX SDK. The only place the name PAX appears.

Start-up

`TerminalFactory`

The only place that makes a `PaxTerminal`

`VendorApplication`

PAX's Application (the SDK needs `BaseApplication`)

`PaxHardwarePermissions`

ICC / Mag / PICC reader permissions

`PaxSetup`

Country and currency; the first-start seed

Card and EMV

`PaxTerminal`

`extends PosTerminal`: the real host and printer, setup

`PaxCardSearch`

The card search (polling): the `SEARCH_CARD` step's work (`onSearchCard`); `stop()` on cancel

`PaxEmvBehavior`

`extends AbstractEmvBehavior`: the 16 `onXxx` (one native call per step), the kernel callbacks, `field55()`

`PaxKernelSession`

What is known during the current attempt; new for each attempt

`PaxKernel`

The kernel objects: params, contact, contactless, mag

EMV helpers

`PaxPinEntry`

The PIN: the pad, the online PIN key (AES) wrapped with RSA for the host, the bypass rules. The host's public key comes from `PaxTerminal`, so the EMV never reads host state itself

`PaxHostAnswer`

The host's answer in the shape the kernel takes for the 2nd GENERATE AC (`toOnlineResult`)

`PaxReaders`

Closes each reader (mag, ICC, PICC) on its own

`PaxCardData`

Card data as text: the PAN from track 2 or tag 5A, PAN masking, text tags (static, tested)

Screens and devices

`PaxPinPad`

The PIN screen

`PaxAppSelectDialog`

Application choice

`PaxPrinter`

The thermal printer

`PaxEmvParams`

The EMV Params screen's data (cache / database / download)

VENDOR INGENICO `:vendor-ingenico`

The same interfaces, as a stub: a fake card and result until the SDK is added. Adding it will change neither the CORE nor the UI.

`TerminalFactory / VendorApplication`
Start-up

`IngenicoTerminal`
`extends PosTerminal` : a fake host and printer

`IngenicoEmvBehavior`
`extends AbstractEmvBehavior` : a fake search (`onSearchCard` ; later Tetra / Axium); each step moves to the next

EMV LIBS (PAX)

`vendor-pax/libs/sdk — emvlib · emvbase · emvservice · bizlib · poslib · commonutils · vendor-pax/libs/ours — emvflow · bizentity`

PAX's SDK and the certified native kernels, all inside the vendor in `vendor-pax/libs/` : `sdk/` is PAX's own code, `ours/` is ours (`emvflow` , `bizentity`). Only vendor-pax uses them (Gradle paths like `:vendor-pax:libs:sdk:poslib` , shortened here to `:poslib`). Changed only for a clear EMV reason.

Kernels

`:emvlib:dpas — ContactProcess`
The contact kernel: from `EMVAppSelect` to `EMVCompleteTrans` ; `EmvCallBackListener`

`:emvlib — ClssProcess + Clss*Process`
Contactless: a kernel per scheme (PayPass, PayWave, ...)

`:emvlib:base / :emvbase`
Base classes, params, results, answers

The device

`:emvflow`
EmvFlowRuntime (DAL), EmvDeviceImpl (APDU, PIN, trace)

`:emvservice`
PinService, MagCardService

`:poslib / :commonutils / :bizlib`
PAX device tools

Params

`:bizentity`
GreenDAO (the database), `EmvParamService` (the cache), the XML parsers, `EmvParamUpdater` , `ConfigInit` (first start)

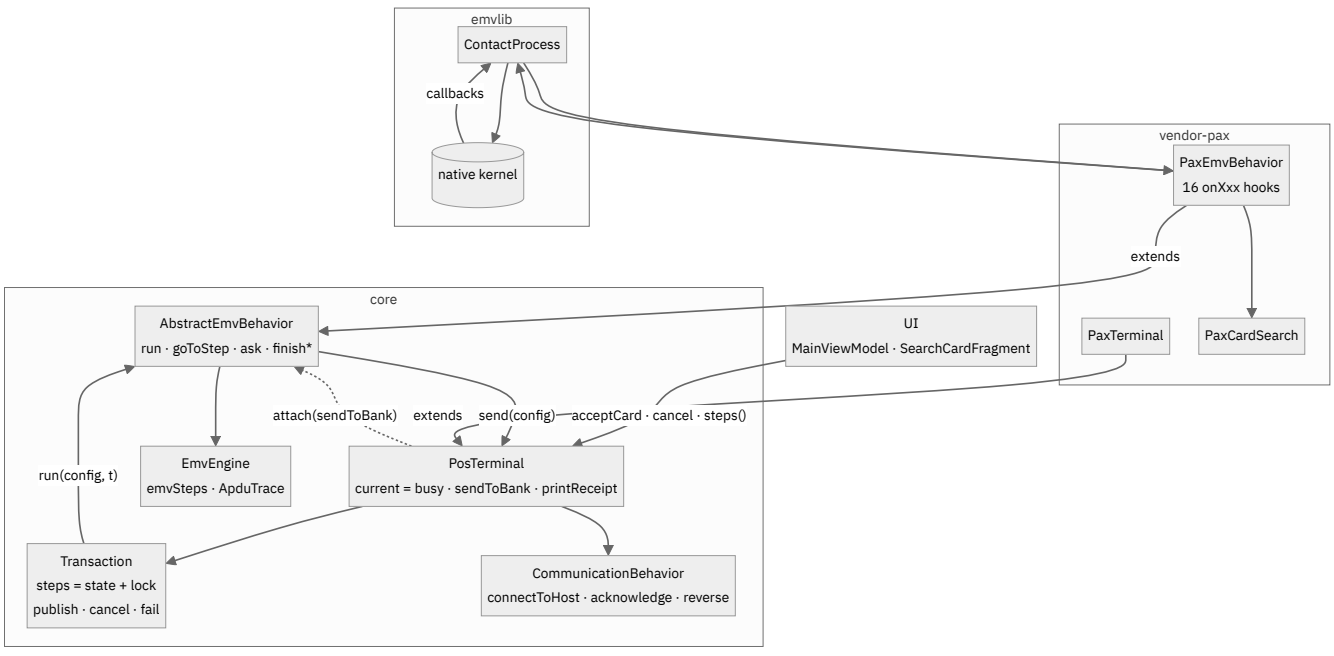
`PosTerminal` is the machine

```
// PosTerminal = the POS machine
AbstractEmvBehavior   behavior;       // the card: search, reader, kernel
CommunicationBehavior communication;  // HTTP to the bank
PrinterBehavior       printer;        // the receipt
Scheduler             transactionScheduler;
                      // one sale at a time, on one thread
```

Class	Owns	Must not
`PosTerminal`	One sale at a time (`current` = busy), the call to the bank (`sendToBank`), the printer.	Hold any EMV logic.
`PosTerminal.Transaction`	The sale's state, read from its steps (`decision()`); the steps themselves; cancel; the requests sent to the bank and their safe closing.	Publish anything after its end.
`AbstractEmvBehavior`	The shared EMV: attempts, `goToStep` , `ask` , `askTheBank` / `bankAnswer` , and the 16 `onXxx` hooks a vendor writes.	Know about threads, the busy state, or call the bank directly.
`PaxEmvBehavior`	The `onXxx` hooks on PAX: one native call per step, and the kernel callbacks.	Be called directly (only through `goToStep`).
`EmvEngine`	The EMV step stream (`emvSteps` : one for the terminal, never completes) and the APDU trace.	Hold any transaction state.
`SaleCommunicationBehavior`	The bank: `connectToHost` (token → exchange → sale / refund), acknowledge, reverse. Decides whether a request reached the bank.	Decide to acknowledge or reverse.

Who calls whom

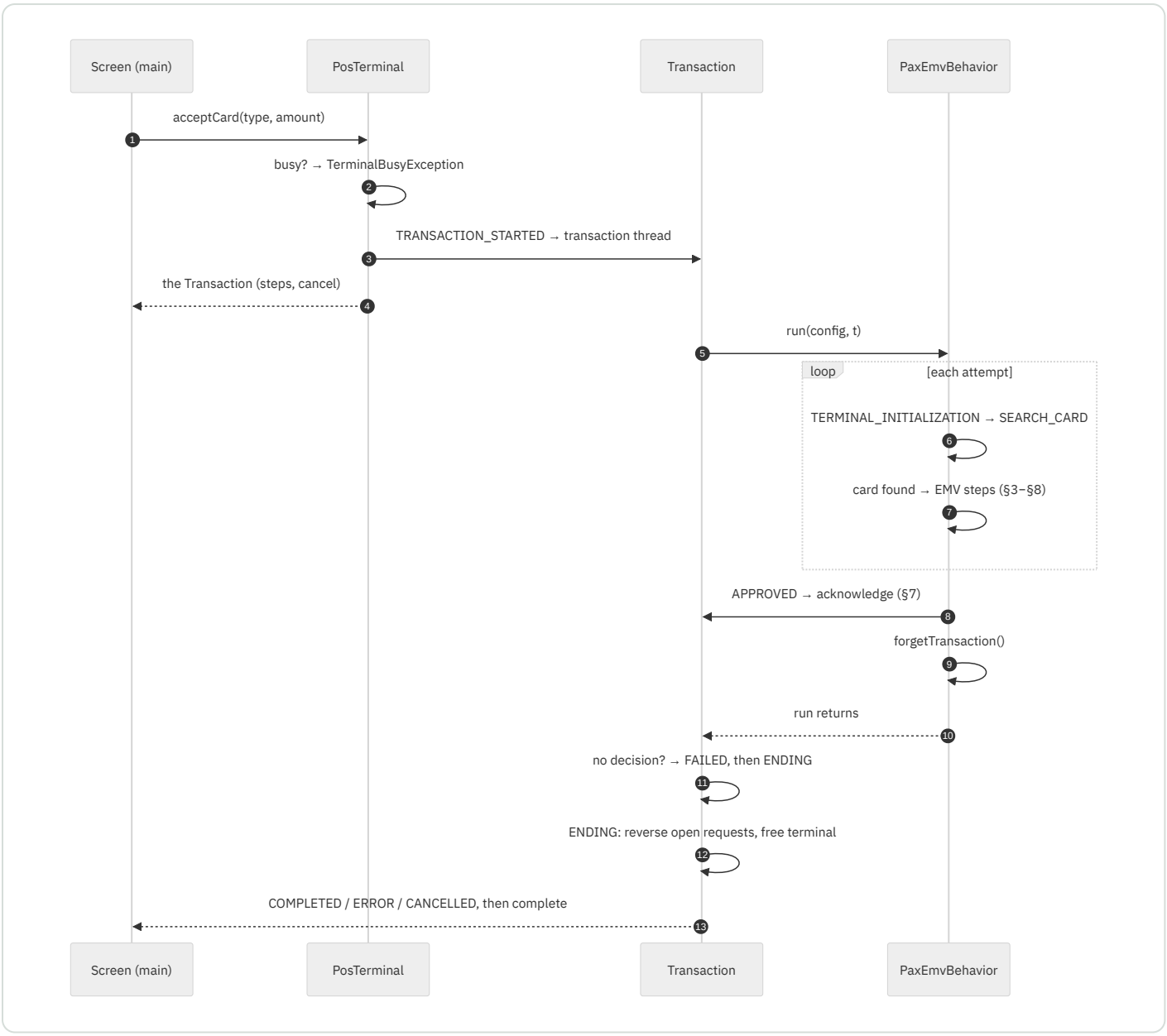
A solid arrow is a real call. A dashed arrow is wired once, at start-up.



2. A sale from start to end

A sale runs on its own steps (`TransactionStep`). Every step enters through one method, `publish` , and is stored in `Transaction.steps` . `TransactionStepObserver` then runs the step's case in `dispatchTransactionStep` . A step with no case is only a report for the screen.

```
// TransactionStepObserver.onNext → dispatchTransactionStep(event)
switch (event.getStep()) {
  case TRANSACTION_STARTED: onTransactionStarted(event);
      // move to the transaction thread
  case CANCELLING: onCancelling();
      // stop the card search
  case APPROVED:   closeLast(ACKNOWLEDGING);
      // bank and card approved → acknowledge
  case DECLINED:   closeLast(REVERSING);
      // bank approved, card declined → reverse
  case ENDING:     onEnding();
      // reverse what is still open, free the terminal, deliver the end
  default:
      // a report: ONLINE_*, CARD_*, FAILED, the closings, the end
}
```

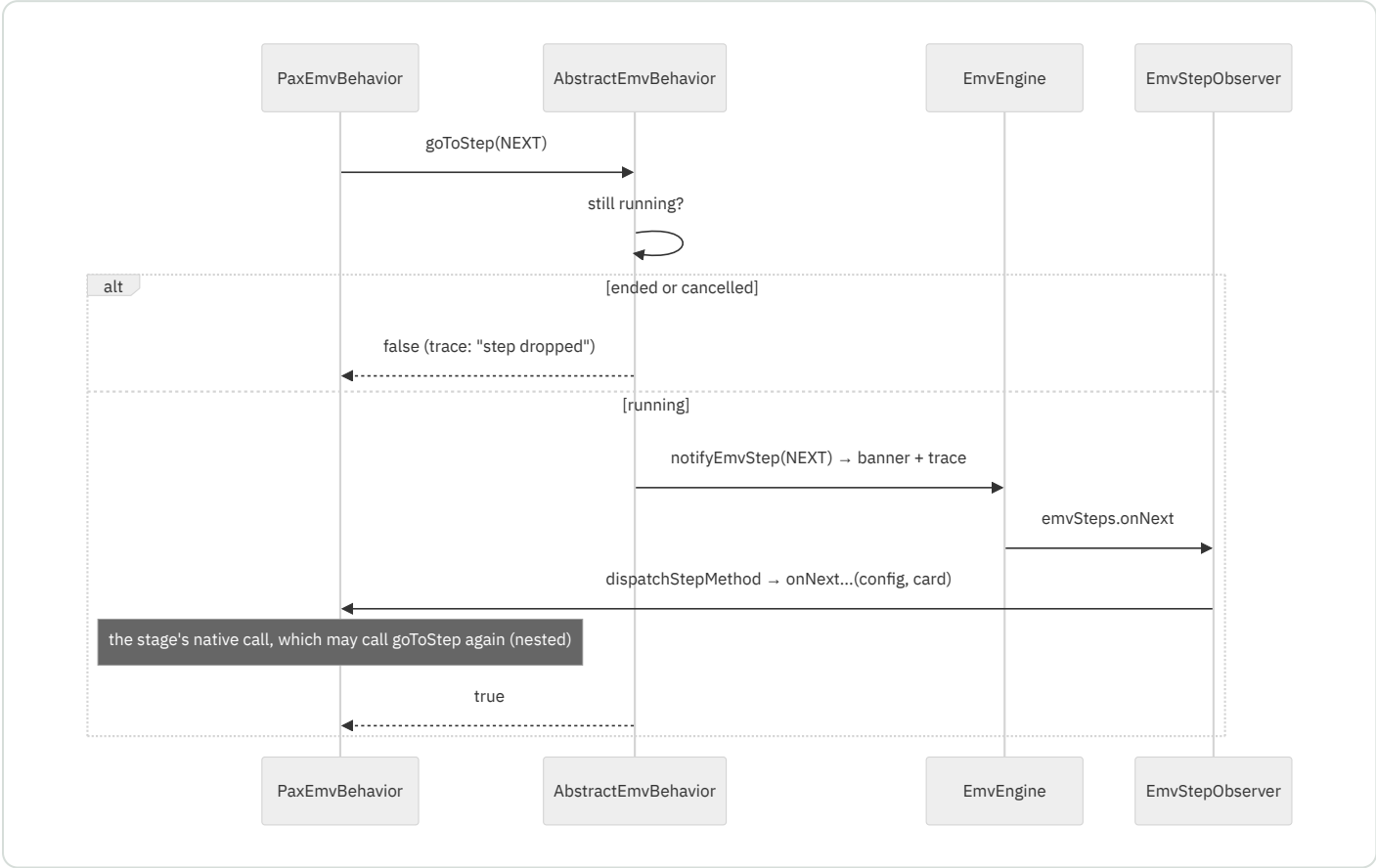


- **Busy.** Checked on the caller's thread. A second sale is refused at once with `TerminalBusyException`; nothing is queued.
- **The state is the steps.** There is no state field. The first decision step delivered is the state (`decision()`): `CANCELLING`, `APPROVED`, `DECLINED` or `FAILED`. No decision means the sale is running. The first decision wins; later ones are dropped.
- **The end follows the decision.** `CANCELLING` → `CANCELLED`; `APPROVED` / `DECLINED` → `COMPLETED`; `FAILED` → `ERROR` with the same message. `FAILED` exists because `ERROR` must stay the last step.
- **Cancel.** `cancel()` publishes `CANCELLING`. The card search stops, and the EMV stops at its next `goToStep`. The thread is never interrupted.
- **Retry** (fallback, tap again, See Phone). The EMV calls `requestRetry(...)` and the kernel returns. `run` then starts a new attempt from `TERMINAL_INITIALIZATION`, in the same sale.

- **The result is not the end.** `APPROVED` / `DECLINED` / `FAILED` are published as soon as they are known. `COMPLETED` comes later, after the bank is acknowledged or reversed and the terminal is free.
- **Nothing stays attached after the end** (`213762c`):
  - The steps complete after the end (`steps.onComplete()`). Every subscriber is released without having to dispose. The steps stay readable, so a late subscriber still receives all of them.
  - `run` ends with `forgetTransaction()` . The EMV dispatcher is disposed, and the sale's data is cleared: the config (with field 55), the card, the bank answer.
  - `emvSteps` never completes. It belongs to the terminal, not to the sale; completing it would block the next sale. The screen listens to it per sale, until the sale ends (`takeUntil`).
- A typical run on screen: `ONLINE_REQUIRED → ONLINE_PROCESSING → ONLINE_COMPLETED → APPROVED → ACKNOWLEDGING → CLOSING_COMPLETED → ENDING → COMPLETED` .
- ⚠ The step cases run while the steps lock is held, including the bank calls of the closings. `publish` therefore checks the decision before taking the lock, so `cancel()` never waits for a closing. Steps that may wait for the bank or a person (`ONLINE_PROCESSING` , `SINGLE_TAP_PIN_REQUIRED`) stay reports, with no case.

3. EMV steps: how `goToStep` runs `onXxx`

Nobody calls an `onXxx` hook directly. `goToStep(STEP)` publishes the step on `emvSteps`. `EmvStepObserver` runs its hook on the same thread, and the hook finishes before `goToStep` returns. If the sale has ended or was cancelled, the step is dropped and `goToStep` returns `false`.



- **Start.** `TERMINAL_INITIALIZATION` prepares the kernel, then calls `searchCard()`, which starts `SEARCH_CARD`. If preparation fails, the sale ends with `ERROR` and no search runs.
- **The dispatch must stay synchronous:** no `observeOn` and no serialize on `EmvStepObserver`. The table below explains why.
- **The screen is the exception.** It may use `observeOn(main)` on `emvSteps`, because it only shows the banner and nothing waits for it. It subscribes per sale, so it can miss the first EMV step; the dispatch is not affected.

Why the dispatch must stay synchronous

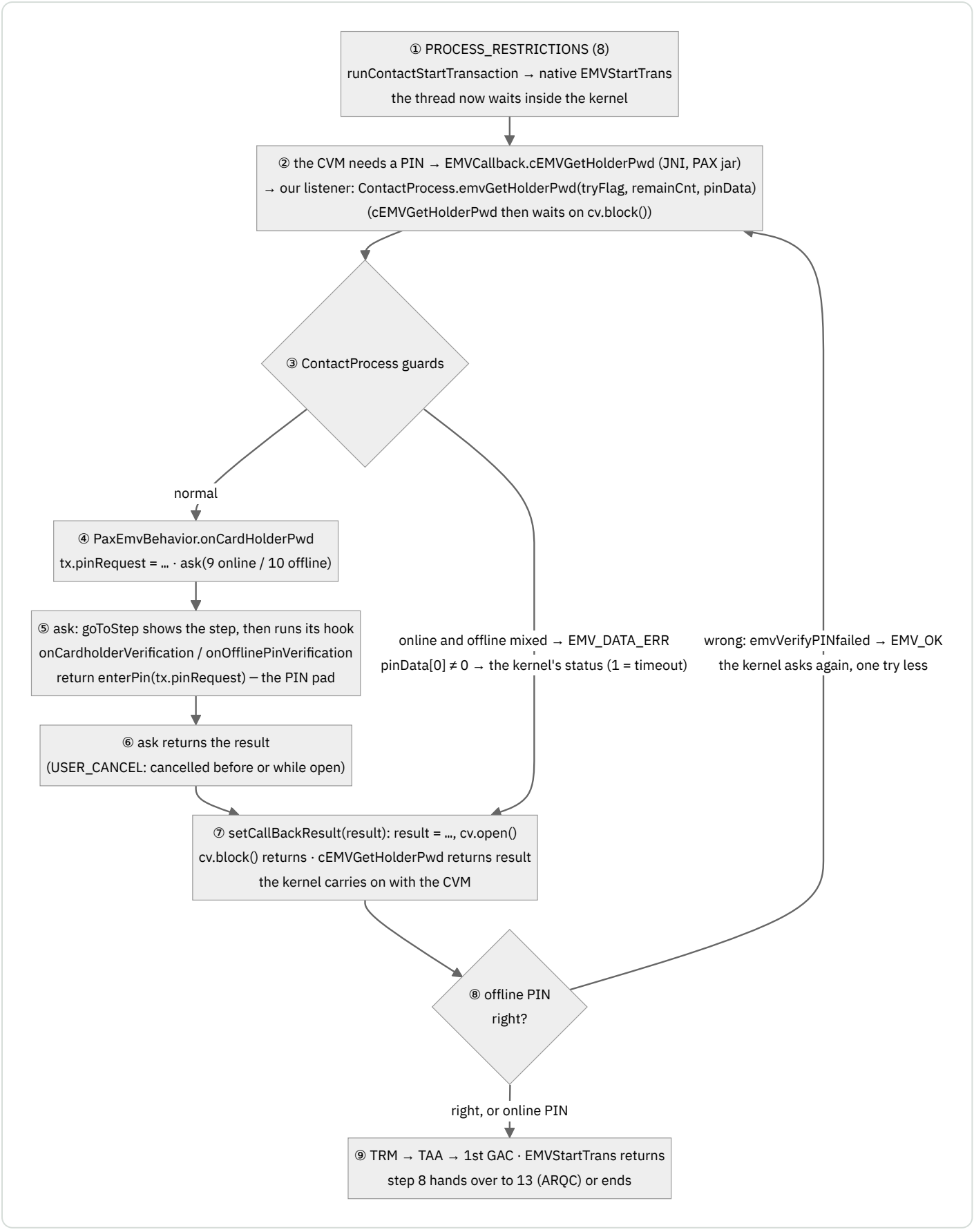
`observeOn(X)` queues `onNext` and runs it later on another thread, so `goToStep` returns at once and the hook runs elsewhere. On `EmvStepObserver` that breaks the flow:

If we use	What happens	Why
`observeOn` on the observer	Two threads call one kernel: a hook calls a native function (`EMVReadAppData` , `EMVCardAuth`) while the transaction thread is still inside another, or `finishContactStage` closes the readers under it.	The PAX kernel and readers are not thread-safe. Everything must run on one thread, in order.
`observeOn` on the observer	The order breaks: `runContactAppSelect` continues as if the read stage had finished and stored its result in `tx.transResult` .	Each stage reads the previous stage's result after `goToStep` returns.
`emvSteps.toSerialized()`	A nested `goToStep(NEXT)` is queued until the current `onNext` returns.	Every hook calls `goToStep` from inside `onNext` , so the stage would find the next stage's result missing.

4. When the kernel waits for a person: `ask`

During one native call the kernel can stop and ask a question through a callback: choose the application, enter the PIN. It waits until it gets an answer (`setCallBackResult`). The callback calls `ask(step, detail, failClosed)` : the step is shown, then its hook asks the person and **returns** the result. That hook runs **inside** the step that called the kernel; the result goes back to the kernel, which carries on from where it stopped. This is the circle:

The PIN circle (contact)

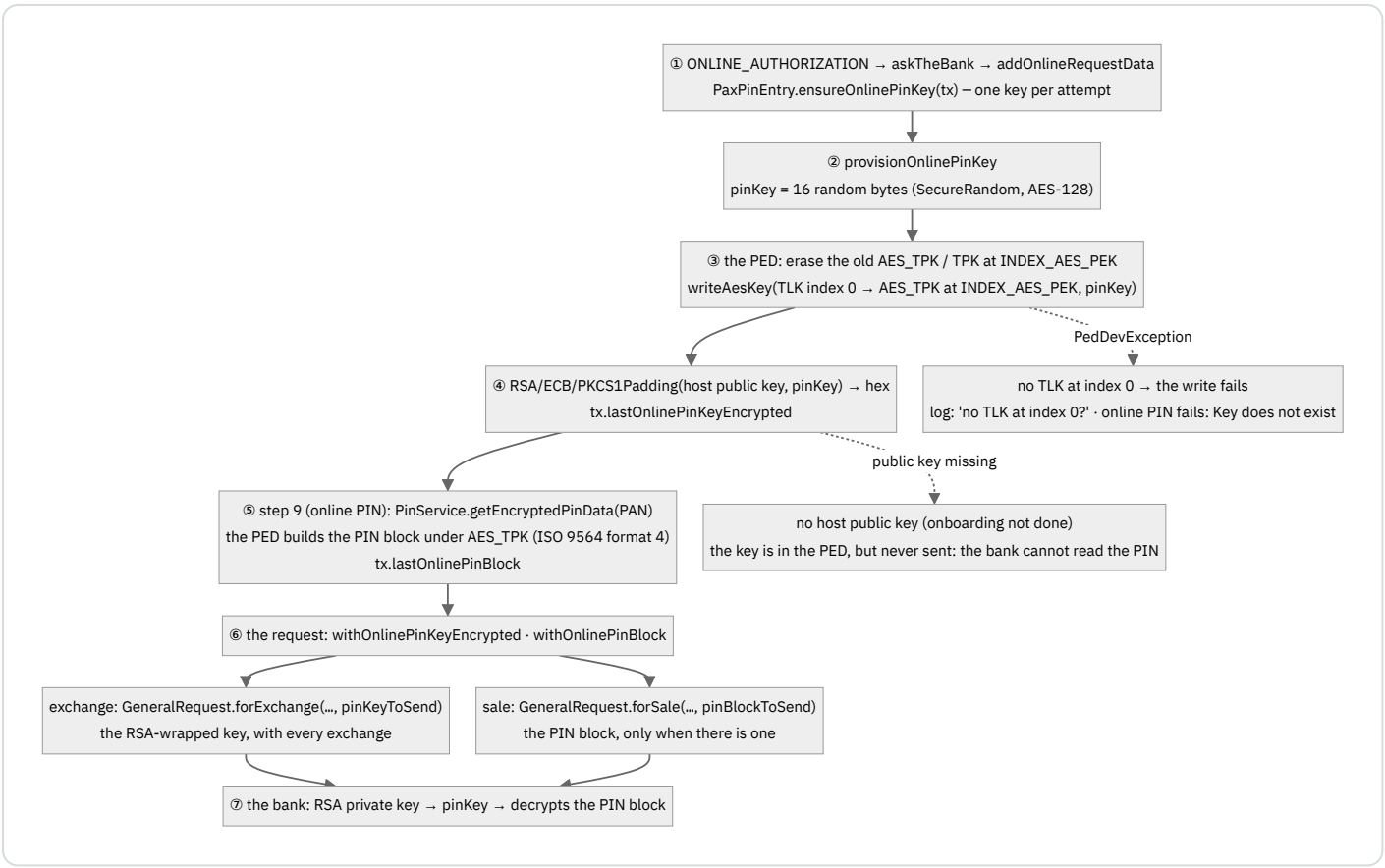


- **The circle is synchronous.** Steps ①–⑨ run on the transaction thread, nested inside each other, and close from the inside out.

- `ask` **is three checks:** the step shown? (else `USER_CANCEL`, no PIN pad) → the hook's result → the sale still running? (else `USER_CANCEL`). The hooks of steps 3 / 4 / 9 / 10 return `int` and are run by `ask` only; the dispatch skips them, so a hook never runs twice.
- **The hook's data.** A hook only gets `(config, card)`, so the callback leaves the PIN request (`tx.pinRequest`) or the candidate list (`tx.candidates`) in the session right before `ask`.
- **Fail closed.** A cancel → `USER_CANCEL`. A hook that throws → `ContactProcess` catches it and answers `USER_CANCEL`. The kernel never waits forever and never approves without an answer.
- **Online PIN** asks step 9 (`CARDHOLDER_VERIFICATION`): PIN pad + PIN key; the bank checks the PIN. **Offline PIN** asks step 10 (`OFFLINE_PIN_VERIFICATION`) directly (`93a798a`): PIN pad only; the kernel checks the PIN with the card.
- **Application selection** is the same circle: `EMVAppSelect` → `emvWaitAppSel` → `onWaitAppSelect` → `ask(3 WAIT_ / 4 FINAL_APPLICATION_SELECTION)` → the hook returns `chooseApplication(tx.candidates)`: the dialog's index, or the only application.
- **Contactless** has no PIN callback: the kernel returns first, then our code calls `onCardHolderPwd(true, …)` itself when the CVM is online PIN (and again for Single Tap, §8). Same `ask`, same hook.

The online PIN key (RSA)

An online PIN goes to the bank as a PIN block made inside the PED under a fresh AES key. The bank gets that key wrapped with its own RSA public key, so only the bank can unwrap it and read the PIN block.



- **One key per attempt, made before the first online request.** It goes with **every** exchange, PIN or not (`pinKeyToSend`): a Single Tap PIN comes later in a resend that has no exchange of its own, so the bank must already hold the key (§8).
- **The clear key never leaves the terminal unwrapped.** It is written into the PED under the TLK (index 0), and sent only RSA-wrapped (`RsaPublicKeyEncryptor`, PKCS#1 v1.5, the host key in PKCS#1 DER).
- **The PIN block is built by the PED**, never by our code; we only carry the bytes (`tx.lastOnlinePinBlock`, hex in the request).
- ⚠ **The TLK must be injected** at index 0 for the key write to work. Without it, every online PIN fails (`Key does not exist`). How the TLK gets there (key injection / TMK provisioning) is not wired yet.
- ⚠ **No host public key** (onboarding not completed): the key is written but not sent, and the bank cannot decrypt the PIN block.
- Code: `PaxPinEntry.provisionOnlinePinKey` / `enter`, `RsaPublicKeyEncryptor.encryptToHex`, `SaleCommunicationBehavior.pinKeyToSend` / `pinBlockToSend` / `prepareExchangeRequest`. Not device-tested on this branch (form rows 4, 5, 7, 9).

How the kernel waits: `EMVCallback` (PAX's jar)

The native kernel does not call our listener directly. It calls a Java method of `EMVCallback` (`EMV_v105.jar`, read with `javap`), and that method is what waits for `setCallBackResult`:

```
// EMVCallback (PAX, decompiled)
public int cEMVGetHolderPwd(int tryFlag, int remainCnt, byte[] pinData) {
    result = -14;                                   // default, before any answer
    if (listener ≠ null) {
        cv = new ConditionVariable();
        listener.emvGetHolderPwd(tryFlag, remainCnt, pinData);  // ContactProcess
        cv.block();                                 // waits for setCallBackResult
    }
    return result;                                  // back to native EMVStartTrans
}

public void setCallBackResult(int value) {
    result = value;
    cv.open();                                      // releases cv.block()
}
```

- **What waits is `cv.block()`** inside `EMVCallback`, not our code. We call `setCallBackResult` on the same thread before the listener returns, so the gate is already open and `block()` returns at once.
- **Never forget `setCallBackResult`:** without it `cv.block()` waits forever and the terminal hangs. That is why every callback in `ContactProcess` calls it on every path.
- **A listener that throws** never reaches `cv.block()`, and the exception goes back into the native call. That is why `ContactProcess` catches `Throwable` and answers `EMV_USER_CANCEL` with `setCallBackResult`.
- PAX built it with a `ConditionVariable` so the answer could come from another thread (a dialog on main calling `setCallBackResult` later). We do not use that: we answer on the same thread.
- The default `-14` (returned when no listener is registered) is not named in the jar; its meaning is not assumed here.

The kernel's callbacks: which are used

`ContactProcess.EmvCallBackListener` is made new at each attempt (`preTransProcess`), so its counters (`isFirstCall`, `tmpRemainCount`) start fresh every time. Every callback ends with an answer.

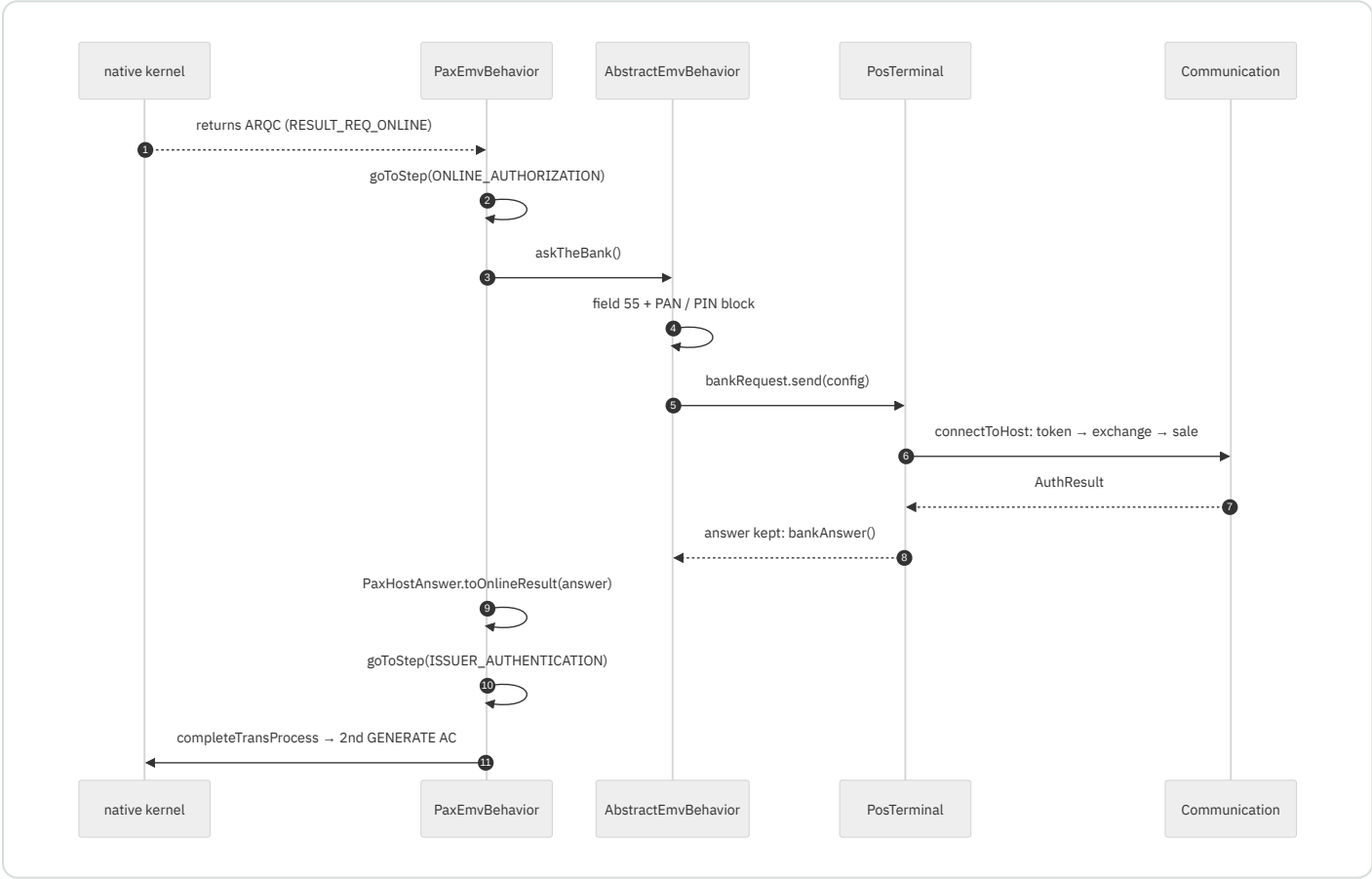
Callback	The kernel calls it	What we do
`emvWaitAppSel`	In `EMVAppSelect`, to choose / confirm the application	**Asks a person:** `onWaitAppSelect` → `ask(3 / 4)`
`emvGetHolderPwd`	In `EMVStartTrans`, when the CVM needs a PIN	**Asks a person:** the circle above
`emvVerifyPINfailed`	The offline PIN was wrong	`EMV_OK`; the kernel decides: it asks again (⓶) or goes on
`emvVerifyPINOK`	The offline PIN was right	Log only
`emvSetParam`	After application selection, before GPO (inside `EMVReadAppData`)	Data: `resetParam(AID)` sets 9F33 / 9F40 / 9F35 and the PIN bypass rules
`emvInputAmount`	The kernel needs the amount	Data: amount and other amount
`emvUnknowTLVData`	The kernel needs a tag it does not have	Data: 9A, 9F1E, 9F21, 9F37, FF01; any other tag → −1
`certVerify`	PBOC cardholder ID check	`EMV_DATA_ERR`: not supported, fail closed
`emvAdviceProc`	Advice	Log only
`cRFU2`	Reserved	Returns 0

Only the two "asks a person" callbacks reach `PaxEmvBehavior`. The data callbacks answer inside `ContactProcess`, with no step.

5. The way to the bank

When the card answers the 1st GENERATE AC with an ARQC, the native call **returns** with `RESULT_REQ_ONLINE`. Our code then calls the bank, and goes back to the kernel for the 2nd GENERATE AC in a later step. Three layers take part:

Layer	Method	Does
EMV	`askTheBank()`, in step `ONLINE_AUTHORIZATION`	Builds field 55 and the card data, asks the bank, keeps the answer (`bankAnswer()`).
Terminal	`PosTerminal.sendToBank(config)`	Names the request (`asyncRequestId`), records it for the safe end, publishes `ONLINE_*`.
Bank	`CommunicationBehavior.connectToHost`	token → exchange → sale / refund, over HTTP.



What the screen sees after the bank

Case	Steps after `ONLINE_COMPLETED`
Bank approved, card approved (TC)	`APPROVED → ACKNOWLEDGING → CLOSING_COMPLETED → ENDING → COMPLETED`
Bank approved, card declined (AAC)	`DECLINED → REVERSING → CLOSING_COMPLETED → ENDING → COMPLETED`
Bank declined	`DECLINED → ENDING → COMPLETED`
No usable answer (`failed`)	`FAILED → ENDING → REVERSING → CLOSING_COMPLETED → ERROR`
Never sent (`notSent`)	`FAILED → ENDING → ERROR`
Bank asks for the PIN (Single Tap, §8)	`SINGLE_TAP_PIN_REQUIRED` → the PIN → `ONLINE_*` again → one of the rows above

- **Two copies of the answer, each with one owner.** `bankAnswer()` is the EMV's: it carries on with the card, and is cleared at each attempt. `OnlineRequest.answer` is the terminal's: it decides the acknowledgement or the reversal. Neither reads the other's copy.
- `askTheBank` **always keeps its answer,** even when it returns early (no field 55, or a cancel), so the vendor reads the real reason.
- **Swipe and manual entry** have no kernel: `onReadApplicationData` goes straight to `ONLINE_AUTHORIZATION`, and the result comes from `bankAnswer()` (`finishNonEmv`).
- `startOnlineProcess()` **is never called.** It is part of PAX's callback interfaces, but neither the kernel nor our code calls it; going online is the `ONLINE_AUTHORIZATION` step. It stays only because the interface requires it.

How the EMV reaches the bank: `attach`

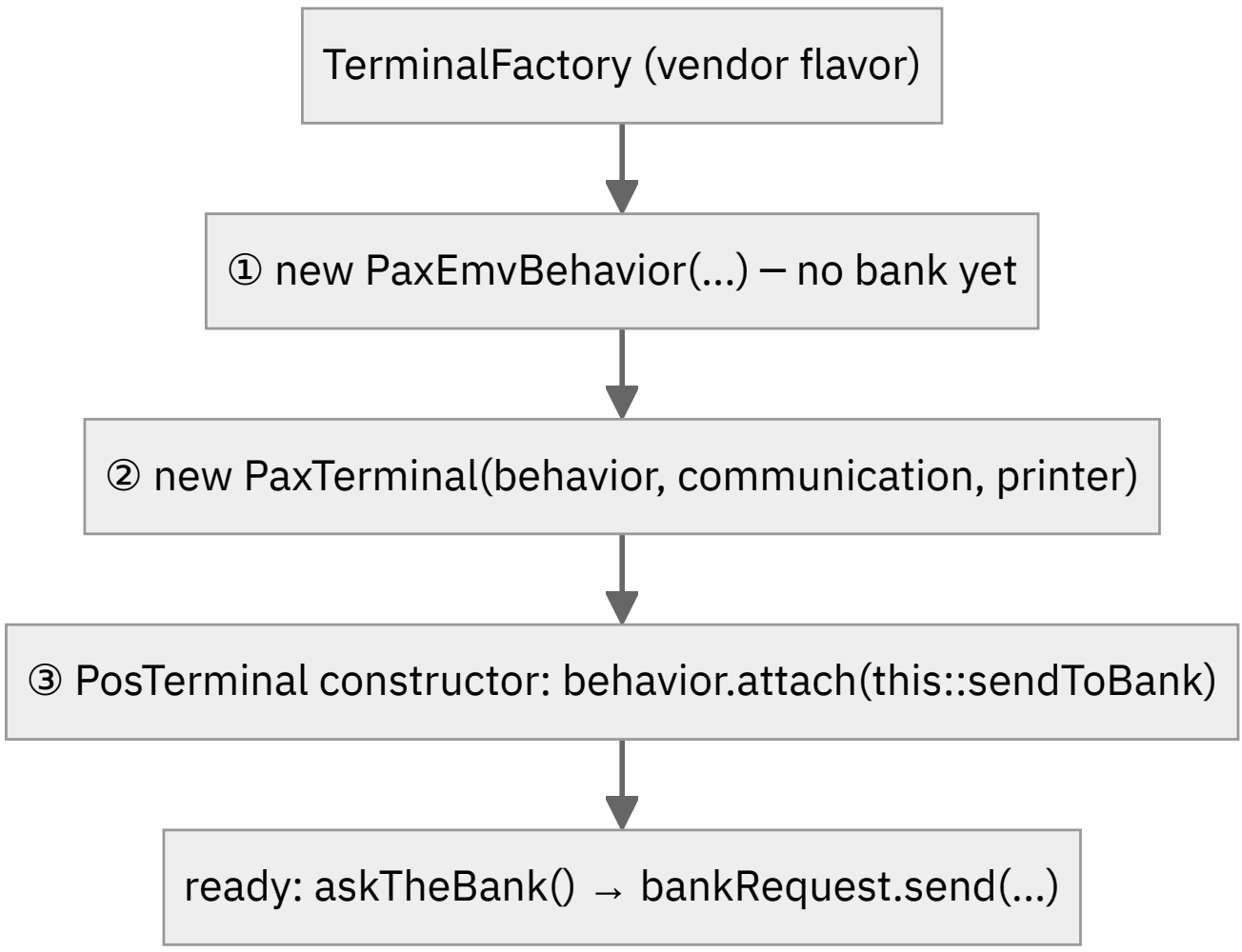
The EMV must call the bank, but must not know `PosTerminal`. So `PosTerminal` gives it one thing to call, a `BankRequest`, once, when it is created:

```
// BankRequest.java
public interface BankRequest { AuthResult send(TransactionConfig config); }

// AbstractEmvBehavior
private BankRequest bankRequest;
public final void attach(BankRequest bankRequest) {
    this.bankRequest = bankRequest;
}
protected AuthResult askTheBank() {
    … return bankAnswer = bankRequest.send(config);
}

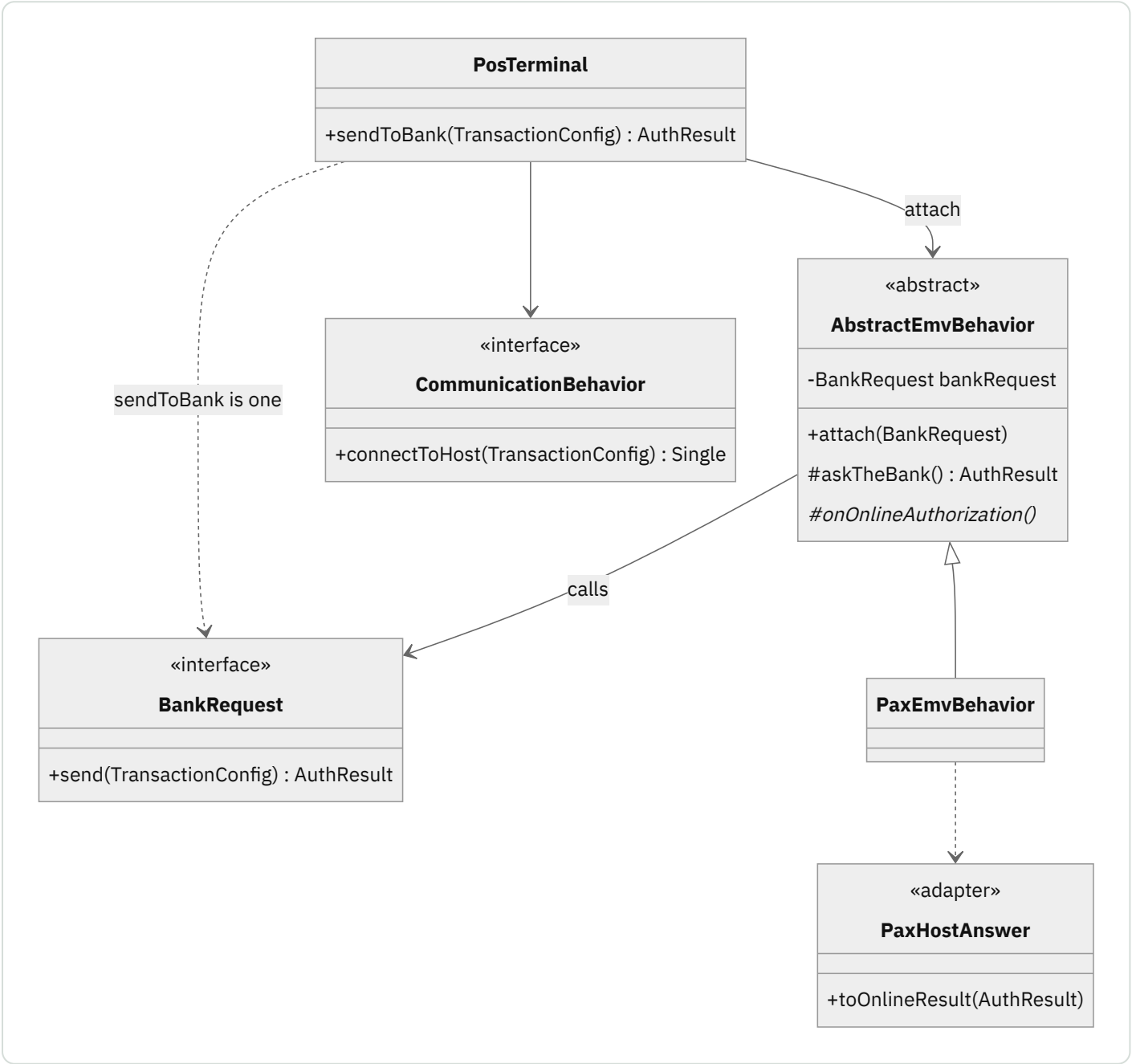
// PosTerminal constructor, once
behavior.attach(this::sendToBank);
```

Why not pass it in the constructor? The two objects need each other. `PosTerminal` needs the behavior in its constructor, and the behavior needs `PosTerminal::sendToBank`, which exists only after `PosTerminal` is built. So the behavior is built first, and wired to the bank right after:



Option	Result
The behavior's constructor takes a `BankRequest`	Not possible: `PosTerminal` does not exist yet.
`PosTerminal` creates the behavior	Rejected: the core would have to know vendor classes.
Dagger / Hilt	Same cycle; its fix (`Lazy` , `Provider` , injection after creation) is `attach` behind annotations. Nothing gets simpler.
`attach` **, once**	**Chosen.** The field is `private` , `attach` is `final` , and it runs before any sale.

The design behind it



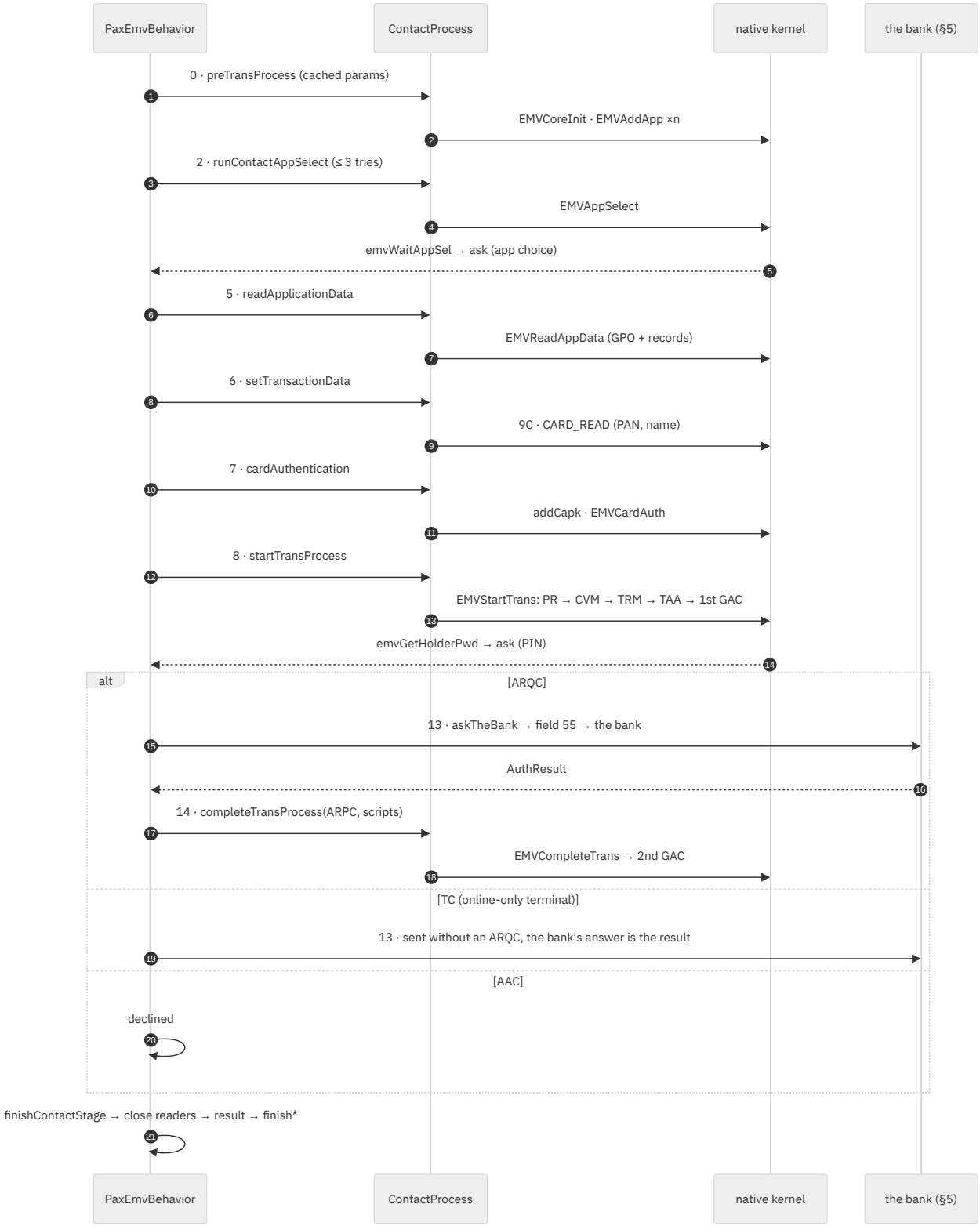
Concept	Where	What it gives us
Dependency Injection (no framework)	Each class gets what it needs from outside: `PosTerminal` its behavior, communication and printer in the constructor; the behavior its `BankRequest` through `attach`.	Each class keeps one job (Single Responsibility).
Dependency Inversion	The behavior depends on the `BankRequest` interface, not on `PosTerminal`.	The EMV knows nothing of the terminal or the bank. A test passes a one-line fake.
Callback via a method reference	`this::sendToBank` is the `BankRequest`.	No extra class implements the interface.
Template Method	`askTheBank()` is shared; `onOnlineAuthorization()` is each vendor's.	Checks, field 55 and the call live in one place; the vendor decides what to do with the answer.
Polymorphism	`CommunicationBehavior`, `AbstractEmvBehavior` (PAX, Ingenico, Fake).	The bank or the vendor changes without touching `PosTerminal`.
Value Object	`AuthResult`: private constructor, factories (`approved`, `declined`, `failed`, `notSent`, `pinRequired`), no setters.	The answer never changes after it is made, and has one vendor-free shape.
Adapter	`PaxHostAnswer.toOnlineResult(AuthResult)`	Turns the shared answer into the shape the PAX kernel reads. Another vendor writes its own.

> All of this is synchronous on the transaction thread: `askTheBank` waits for `sendToBank` to return its `AuthResult`, then the code carries on with the card.

6. The contact path (in the kernel's own order)

One stage = one native call = one `EmvStep`. Each `runContact*` method calls the kernel. On success it moves on with `advanceContactStage(NEXT)`; otherwise it ends the path with `finishContactStage()`, which releases the listener, closes the readers and reads the result.

#	EmvStep	PAX method	Native call	Asks a person?
0	`TERMINAL_INITIALIZATION`	`prepareKernel` → `preTransProcess`	`EMVCoreInit`, `EMVAddApp`	—
1	`SEARCH_CARD`	`onSearchCard` → `PaxCardSearch`	polling	—
2	`APPLICATION_SELECTION`	`runContactAppSelect`	`EMVAppSelect`	Application choice
5	`READ_APPLICATION_DATA`	`runContactReadAppData`	`EMVReadAppData` (GPO + records)	—
6	`SET_TRANSACTION_DATA`	`runContactSetTransactionData`	9C, `CARD_READ`	—
7	`OFFLINE_DATA_AUTHENTICATION`	`runContactCardAuth`	`EMVAddCAPK`, `EMVCardAuth`	—
8	`PROCESS_RESTRICTIONS`	`runContactStartTransaction`; then 13 if the card goes online, else the stage ends	`EMVStartTrans` = PR + CVM + TRM + TAA + 1st GAC	PIN
9 / 10	`CARDHOLDER_VERIFICATION` / `OFFLINE_PIN_VERIFICATION`	Asked from inside step 8 (`ask`, §4)	— (the kernel waits in `EMVStartTrans`)	PIN
13	`ONLINE_AUTHORIZATION`	`onOnlineAuthorization` → `askTheBank` → `runContactOnline`; then 14	— (the bank)	—
14	`ISSUER_AUTHENTICATION`	`runContactIssuerAuthentication`; then the stage ends	`EMVCompleteTrans` = ARPC + scripts + 2nd GAC	—



- **When the sale goes to the bank.** When the card answers with an ARQC; a refund asks for an ARQC like a sale. The exception: a TC (`RESULT_OFFLINE_APPROVED`) on this online-only terminal is sent without an ARQC, and the bank's answer is the result. An AAC is a decline.
- **Each step does its own work, then hands over.** `PROCESS_RESTRICTIONS` → `ONLINE_AUTHORIZATION` → `ISSUER_AUTHENTICATION`, each in its own hook. `finishContactStage` runs in the last step that works, or where the path stops.

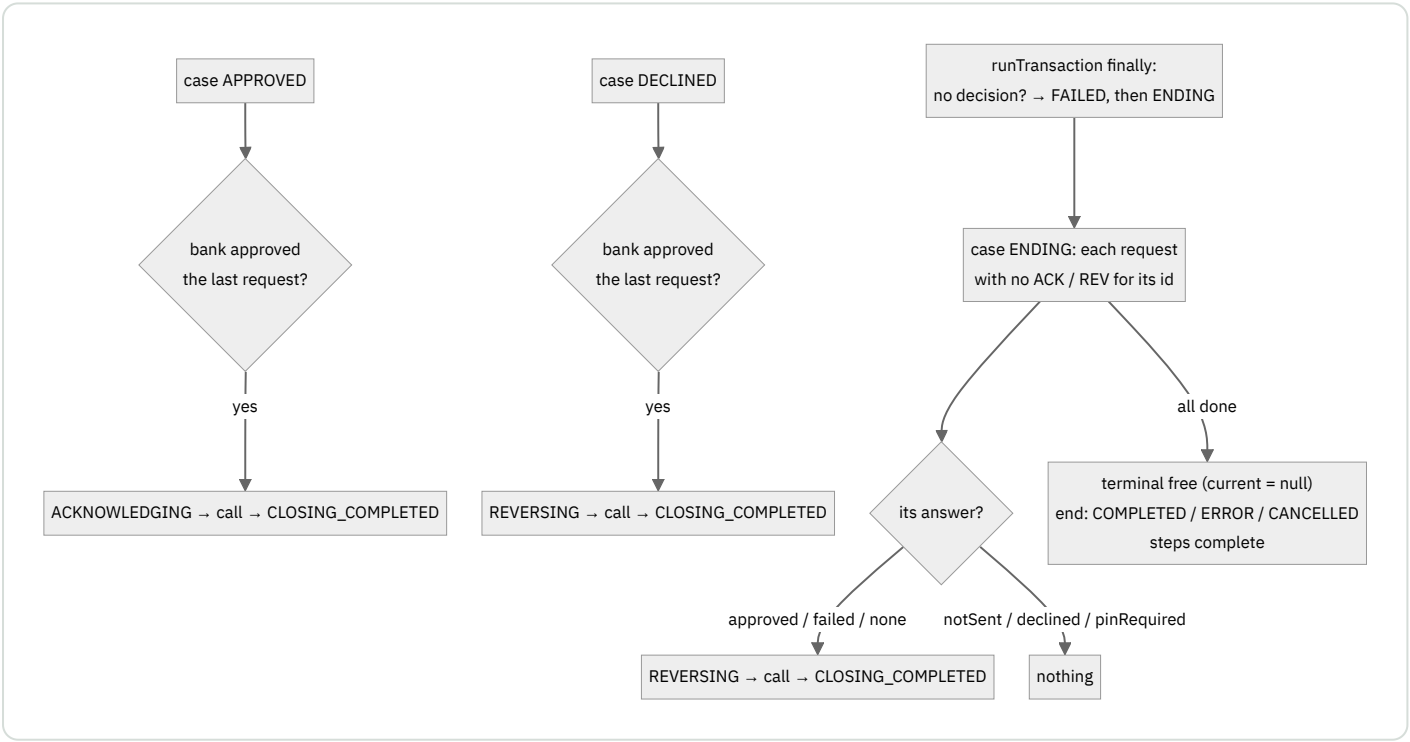
- **The 2nd GENERATE AC always follows the bank** (AET-146), even when the bank declined or did not answer. Only a cancel while waiting for the bank skips it: `goToStep(ISSUER_AUTHENTICATION)` returns `false` and the stage ends.
- **Scripts** run inside `EMVCompleteTrans` (step 14), so contact never publishes `SCRIPT_PROCESSING` (`cf04104`). Step 15 appears only for PayWave's second tap, where the scripts are a call of their own.
- `TERMINAL_RISK_MANAGEMENT` and `TERMINAL_ACTION_ANALYSIS` are not published: they have no native call of their own (they run inside `EMVStartTrans`).
- **The outcome** is reported once: `completeApproved` / `completeDeclined` → `finishApproved` / `finishDeclined` (no completion step).

7. The outcome and the safe end

The EMV reports the outcome (`finishApproved` / `finishDeclined` → `publish(APPROVED / DECLINED)`; `finishError` → `fail`). The `Transaction` ends the sale. Before the terminal is free again, every request sent to the bank is closed: **acknowledged** or **reversed**.

Who closes what

- **The decision closes the last request.** `case APPROVED` acknowledges it and `case DECLINED` reverses it, but only if the bank approved it. At that moment the bank's answer is known, the card has decided, and the reader is closed.
- `ENDING` **closes the rest.** Every request with no `ACKNOWLEDGING` / `REVERSING` step carrying its id is reversed if the bank may have approved it (approved, `failed`, no answer). This covers a cancel (it can arrive while the bank is still answering), `FAILED`, and requests from an earlier attempt. Then the terminal is freed and the end is delivered.
- **The id links them.** `sendToBank` names each request (`asyncRequestId`) and records it with its answer (`wentOnline`) before sending. A Single Tap resend is the same request, so it is closed once.



Every outcome

Bank	Card (2nd GAC)	Kernel result	End	At the bank
Approved	TC	`ONLINE_APPROVED`	APPROVED → COMPLETED	Acknowledge
Approved	AAC	`ONLINE_CARD_DENIED`	DECLINED → COMPLETED	Reverse
Declined	—	`ONLINE_DENIED`	DECLINED → COMPLETED	Nothing
No usable answer (`failed`)	AAC	`ONLINE_FAILED`	FAILED → ERROR	Reverse (it may have approved)
Never sent (`notSent`)	AAC	`ONLINE_FAILED`	FAILED → ERROR	Nothing (it never arrived)
Any, then a cancel	—	—	CANCELLED	Reverse if the bank may have approved

- For the card, `failed` and `notSent` are the same (`RESULT_ONLINE_FAILED` , 2nd GAC with AAC). They differ only in the reversal.
- The error names the stage that failed, in the message and the trace: `Online failed: Exchange: HTTP 403 Forbidden` .

Who decides `notSent` : the network only

Where it failed	Answer	Reverse?
Before sending: no refund reference, no EMV result / key / token, encryption failed	`notSent`	No
`Token refresh: …` or `Exchange: …`	`notSent`	No
`Sale:` no connection (`ConnectException` / `UnknownHost`), HTTP 401 / 403	`notSent`	No
`Sale:` timeout, cut connection, 5xx, other 4xx, bad envelope	`failed`	Yes

Timeouts

Timeout	Value	Note
Each payment call (OkHttp `callTimeout`)	25 s	token + exchange + sale ≤ 75 s. Payments only, not the params download.
`PosTerminal.sendToBank`	90 s	Safety net (see ⚠).
`closeAtHost` (acknowledge / reverse)	60 s	Safety net (see ⚠).

- **⚠ The 90 s and 60 s nets do not cut the wait.** `connectToHost` / `acknowledge` / `reverse` are `fromCallable` with no `subscribeOn` , so they run on the transaction thread and `timeout` returns only when the call does (a 500 ms timeout waited 3124 ms). The real bound is `callTimeout` . An answer after the timeout is thrown away as `failed` , so the request is

reversed (safe for the money). The fix (`subscribeOn(io)`) needs a decision: the call would go on in the background.

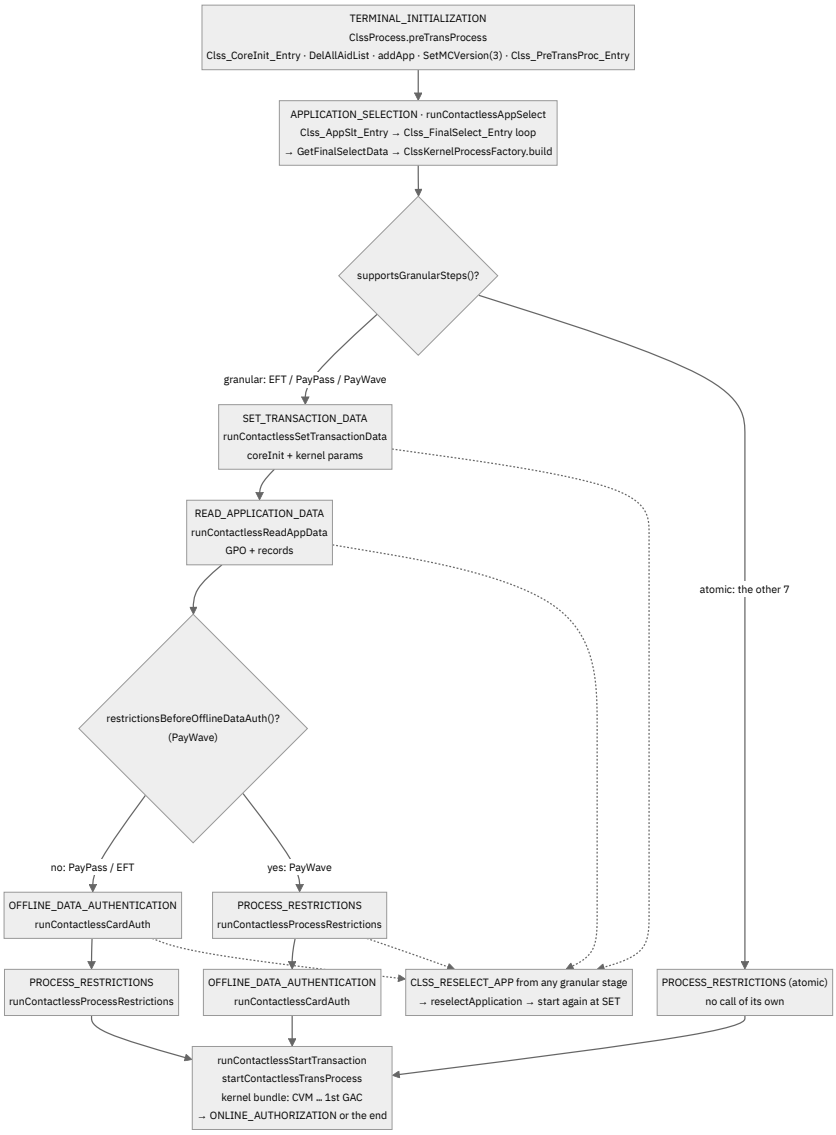
- Payment calls use the synchronous adapter (`createSynchronous()`). Otherwise Retrofit swallows the error when `callTimeout` cuts a call, and the terminal stays busy forever (fixed in `1a97dee`).

Open risks

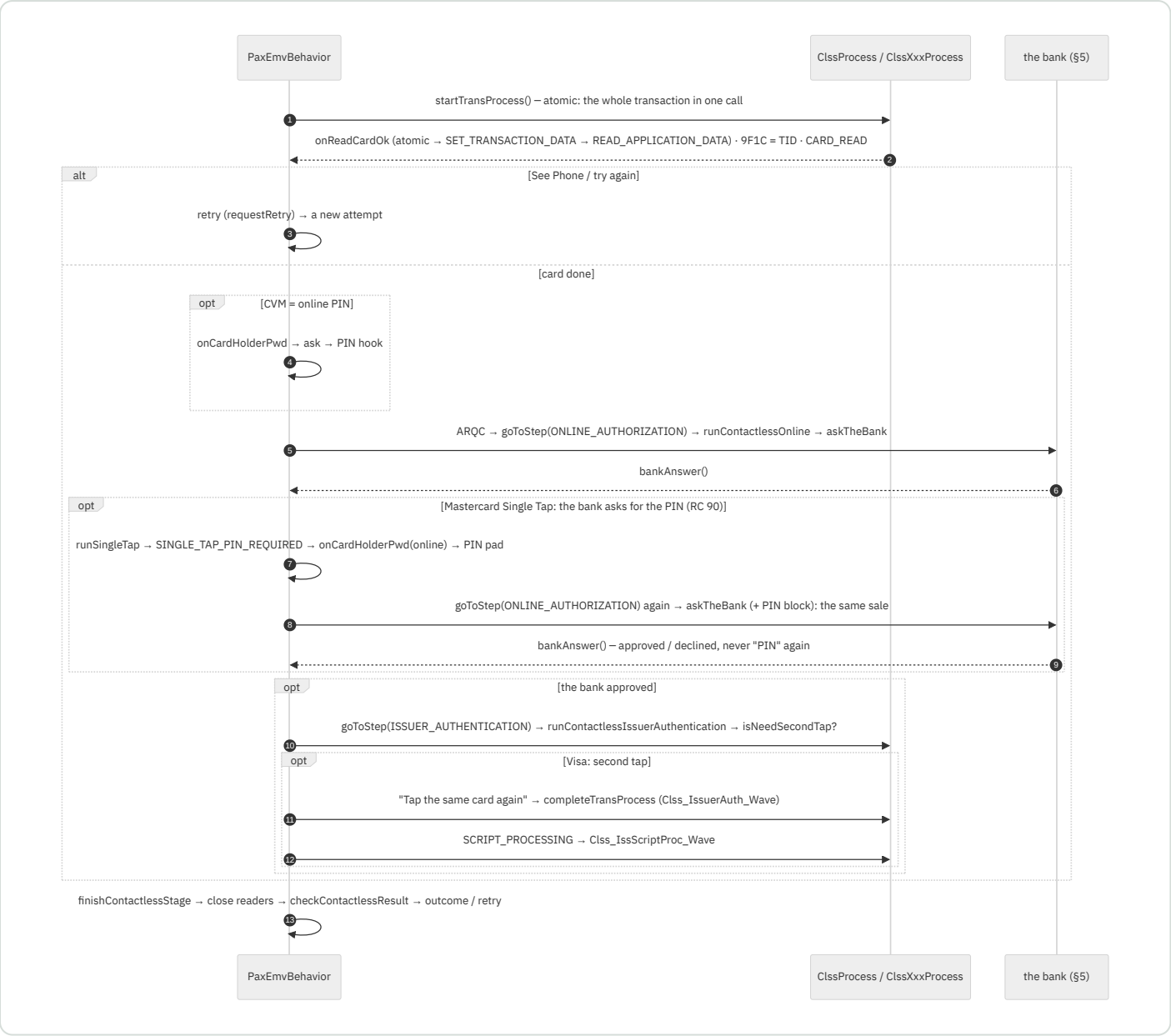
- ⚠ An acknowledgement or reversal is sent once. If it fails, it is only logged; there is no stored journal to resend it (BACKLOG 0.15).
- ⚠ A cancel while waiting for the bank ends the sale CANCELLED, even if the bank approved; that request is then reversed.
- ⚠ Not tried on a device or with the bank. That the closings need the `mToken` is an assumption.
- Trace lines (tag `HOST`): `Acknowledgement <id> sent` or `… failed — not retried: <why>` ; `REVERSAL for request <id>: approved, not completed` ; `no reversal …: the host could not have approved it — <why>` .

8. The contactless path

The same dispatch and the same `ask` as contact, but the kernel is chosen after application selection (`ClssKernelProcessFactory`). There are two kinds: **granular** kernels (PayPass, PayWave, EFT) have one native call per stage; **atomic** kernels (Amex, JCB, Mir, PBOC, Pure, RuPay, DPAS) do everything in one call.



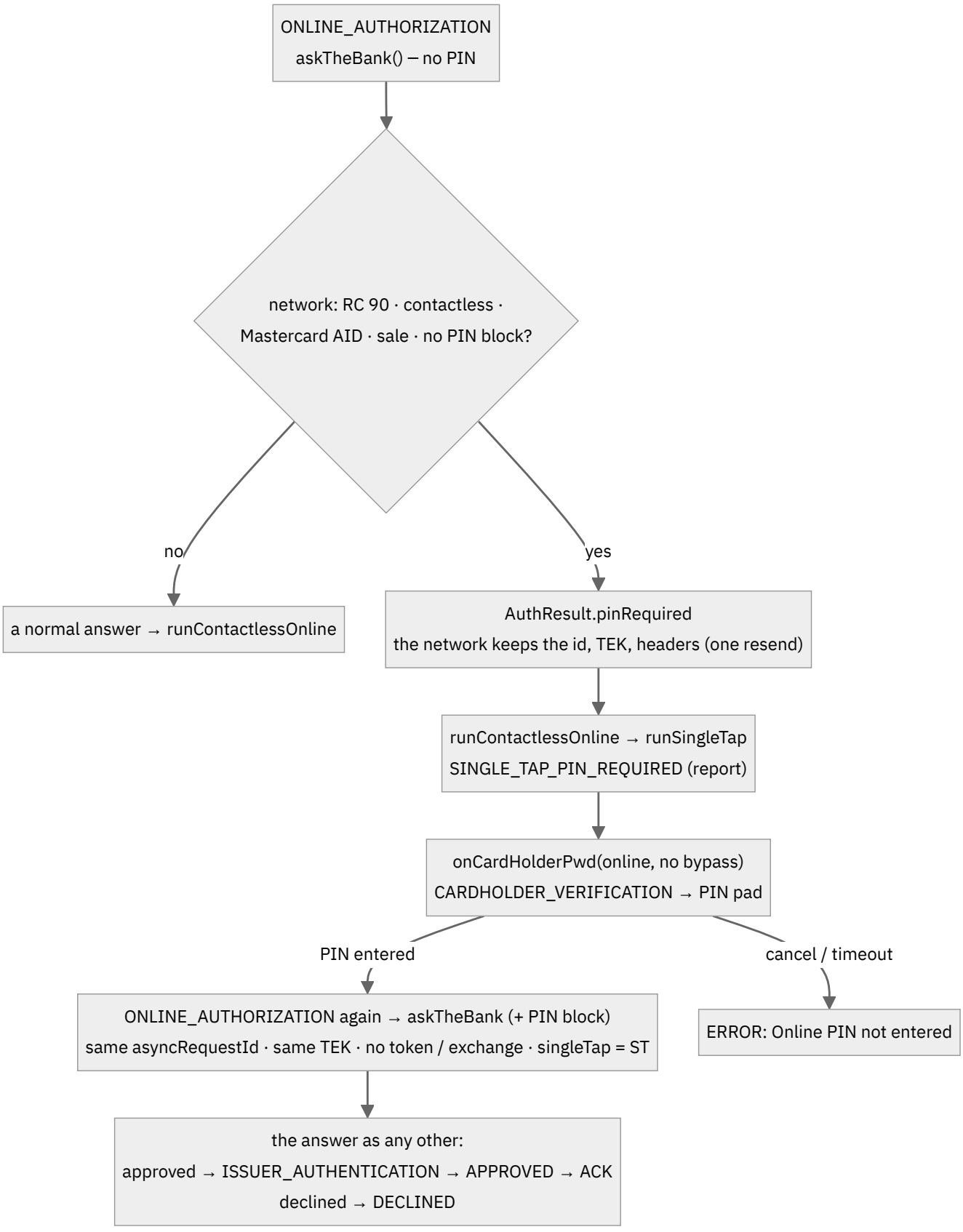
After the stages: `startContactlessTransProcess`



- **The kernel's order wins, not the numbers.** In an atomic kernel, `SET_TRANSACTION_DATA` and `READ_APPLICATION_DATA` are announced from the callbacks at the moment the kernel says they happened. Nothing is read twice; the hooks do nothing for atomic kernels.
- If the bank does not approve, there is no issuer authentication and no 2nd GENERATE AC: the card has left the field.
- ⚠ The second-tap prompt does not wait for the card before `Clss_FinalSelect_Entry` (BACKLOG 1.4).

Mastercard Single Tap: the bank asks for the PIN (`35ce685`)

A contactless Mastercard sale under the CVM limit goes to the bank without a PIN. The bank may answer RC 90 ("PIN required"). The terminal then takes the PIN, **with no second tap**, and sends the same sale again, as the old app did.



- **No loop: three separate guards,** each with a test that fails without it. Network: RC 90 with a PIN is a plain decline, never "PIN required" again. Core: the resend is the same request, so it is closed once, and `pinRequired` is never reversed. PAX: the PIN is asked for once per attempt (`tx.singleTapPinEntered`); a second request ends the sale DECLINED.

- **The PIN key** goes with every exchange, with or without a PIN (`pinKeyToSend`), one key per attempt (`ensureOnlinePinKey` in `addOnlineRequestData`). The resend's PIN block is therefore made under the key the bank got in the first exchange.
- The PIN pad and the resend do not hold the steps lock, so `cancel()` does not wait for them.
- ⚠ **Open:** how does RC 90 arrive, HTTP 200 or 4xx? As 4xx it becomes `failed` + a reversal, and Single Tap never starts (BACKLOG 4.11). The resend's posEntryMode is 071 (an assumption). Not tried on a device or with the bank.

EmvStep	PayPass (`ClssPayPassProcess`)	PayWave (`ClssPayWaveProcess`)	EFT (`ClssEFTProcess`)
`SET_TRANSACTION_DATA`	`Clss_CoreInit_MC`, `SetParam_MC`, FinalSelectData + `setPayPassParam` (DF8117/18/19/1F, limits)	`CoreInit_Wave`, ReaderParam (`aucTmCap` = 9F33) + `SetVisaAidParam_Wave`, `SetTransData_Wave`	`CoreInit_EFT` + ReaderParam + AidParam + `SetTransData_EFT`
`READ_APPLICATION_DATA`	`Clss_InitiateApp_MC` + `Clss_ReadData_MC`	`Clss_Proctrans_Wave` (GPO + records + AC type)	`Clss_InitApp_EFT` + `Clss_ReadAppData_EFT`
Order after it	ODA, then PR	**PR, then ODA**	ODA, then PR
`OFFLINE_DATA_AUTHENTICATION`	Loads the CAPK and revocations only (the ODA runs inside `TransProc`)	`Clss_CardAuth_Wave` (on a TC that is not a refund, or by TTQ)	`PreTAAProc` + CAPK + `Clss_CardAuth_EFT`
`PROCESS_RESTRICTIONS`	No call	`Clss_ProcRestric_Wave`	`Clss_ProcRestrict_EFT`
The bundle	`Clss_TransProc_MC_MChip` / `_Mag` = ODA+PR+CVM+TRM+TAA+GAC, then See Phone	`genTransResult`: `Clss_GetCvmType_Wave` + AC type	`Clss_CVMProc_EFT` + `Clss_PostTAAProc_EFT`
2nd tap / completion	`completeTransProcess`	FinalSelect + `Clss_IssuerAuth_Wave`	`Clss_CompleteOnlineTrans_EFT` ⚠ not called
`SCRIPT_PROCESSING`	Not published	`Clss_IssScriptProc_Wave` (second tap)	Not published

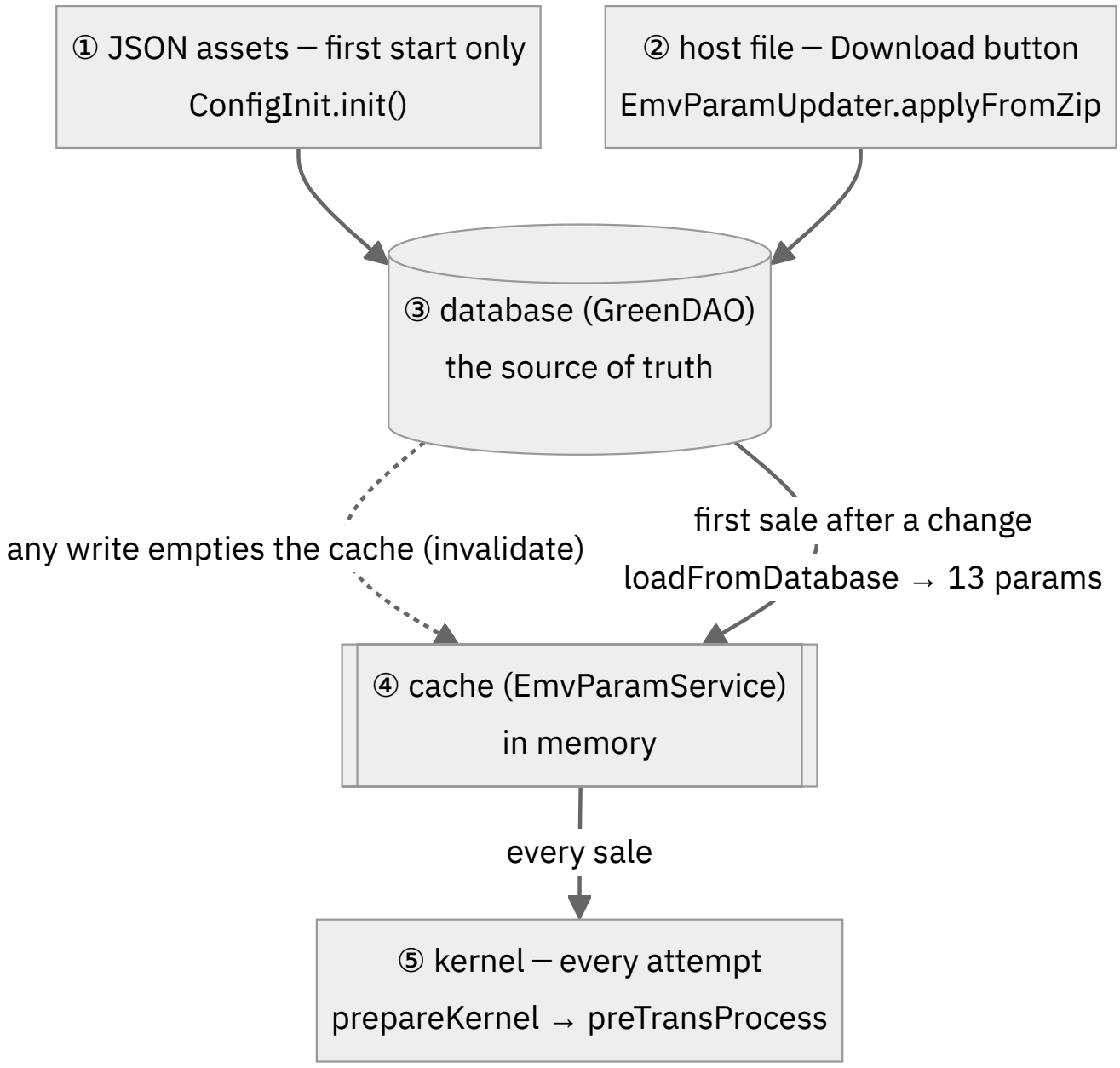
The result: `checkContactlessResult`

Case	Method	End
`RESULT_CLSS_SEE_PHONE` (any ret)	`seePhone` (+ `clsLastNeedSeePhone`)	Retry contactless; declined after 3
`RESULT_TRY_AGAIN` (any ret)	`tryAgain`	Retry the same way; declined after 3
Bank answer `pinRequired` (before the result is read)	`runSingleTap`	The PIN, then `ONLINE_AUTHORIZATION` again
OK + `ONLINE_APPROVED`	`onlineApproved`	APPROVED
OK + `OFFLINE_APPROVED`	`offlineApproved`	ERROR "Online Failed"
OK + anything else	`offlineDenied(CLSS_DECLINE)`	DECLINED
`TRY_ANOTHER_INTERFACE` / `CLSS_USE_CONTACT`	`tryAnotherInterface`	Retry with CHIP
`ONLINE_DENIED` / `ONLINE_CARD_DENIED` / `OFFLINE_DENIED`	As in contact	DECLINED
`ONLINE_FAILED`	`onlineFailed`	ERROR

`IContactlessCallback`	Called by	Does
`onReadCardOk`	The kernel, after the read	Atomic: `goToStep(SET_TRANSACTION_DATA)`, then `goToStep(READ_APPLICATION_DATA)`; `APPLICATION_SELECTED`; 9F1C; `CARD_READ`. It comes before the card decides, so `CARD_READ` also shows before See Phone or a decline.
`confirmCard`	Nobody (the kernel never calls it)	Returns `CONTACT_OK`; kept because the interface requires it
`onWaitAppSelect` / `onCardHolderPwd`	Shared with contact	`ask`
`startOnlineProcess`	Nobody	Returns `bankAnswer()` in PAX shape; going online is `ONLINE_AUTHORIZATION`
`onDetect2ndTap`	The completion stage	A prompt + `SEARCH_CARD` (shown only, no second search)
`needSeePhone`	The kernel on the next tap (Amex / Mir / Pure)	Returns `clsLastNeedSeePhone`, then clears it
`onRemoveCard`	PayPass / PayWave	Logged only

9. EMV params: where they come from, where they live, when they reach the kernel

What they are. The rules the kernel needs to decide each sale: which cards are accepted (AIDs), the keys that prove a card is genuine (CAPKs), and the amount limits (for example, the amount above which a contactless sale asks for a PIN). Without them the kernel cannot run.



1. **First start.** The database is filled once from the JSON files in the app, so there is something to work with.
2. **Download.** The host sends one XML for contact and one for contactless. They are parsed and written over the old values.
3. **The database is the source of truth.** The real values live there.

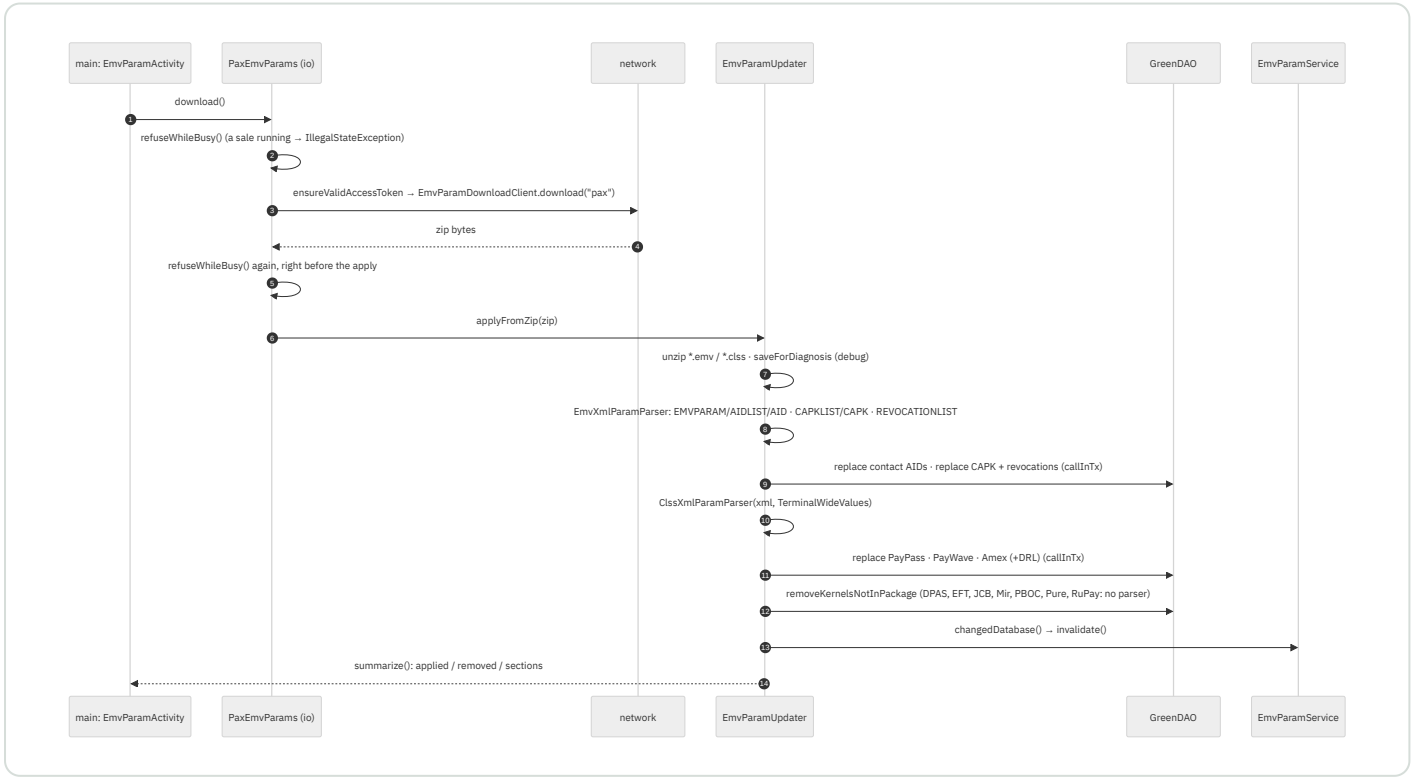
4. **The cache is a fast copy.** The first sale after a change reads the whole database once; later sales read memory.
5. **The kernel keeps nothing.** Every attempt loads the params from the cache again.

Example. The host raises the contactless PIN limit from 600 to 1000 EGP. Download writes 1000 and empties the cache. The next sale reloads the cache (1000) and hands it to the kernel. A tap for 800 EGP now goes through with no PIN.

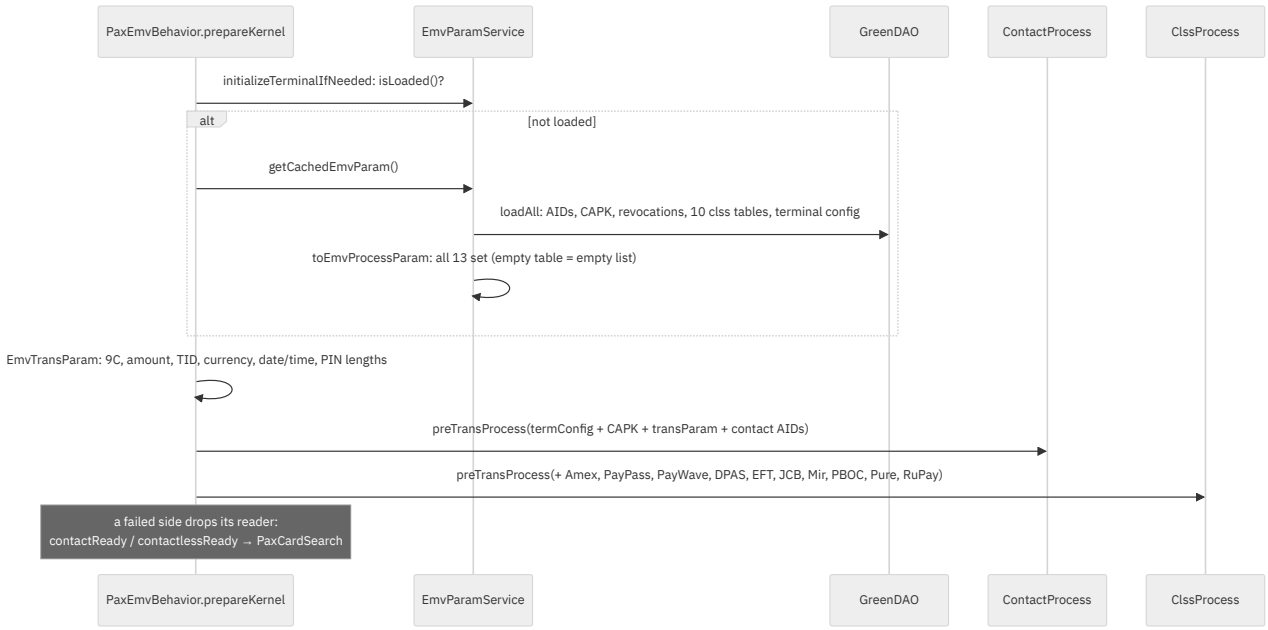
Rules

- **Contact:** what the host file does not bring keeps its old value. **Contactless:** the host file is the whole truth; a kernel it does not bring is deleted (DPAS, EFT, JCB, Mir, PBOC, Pure, RuPay always: no parser).
- Terminal identity (TID, MID, MCC, merchant name) does **not** come from the host file (`OTHERCONFIGURATION` is skipped on purpose).
- **EMV Params screen: DB** shows the stored values; **Cached** shows what the kernel gets now. An empty cache means no sale since the last change, not an error.
- ⚠ CAPKs are not checked (expiry, checksum): 76 of the 88 seed CAPKs have expired. A CAPK with no valid RID or key id is skipped.
- `5755adb` restored the 12 missing `setXxxParam` calls in `loadFromDatabase` (the `preTransProcess` crash); `EmvParamServiceCacheTest` guards it.

The download, step by step



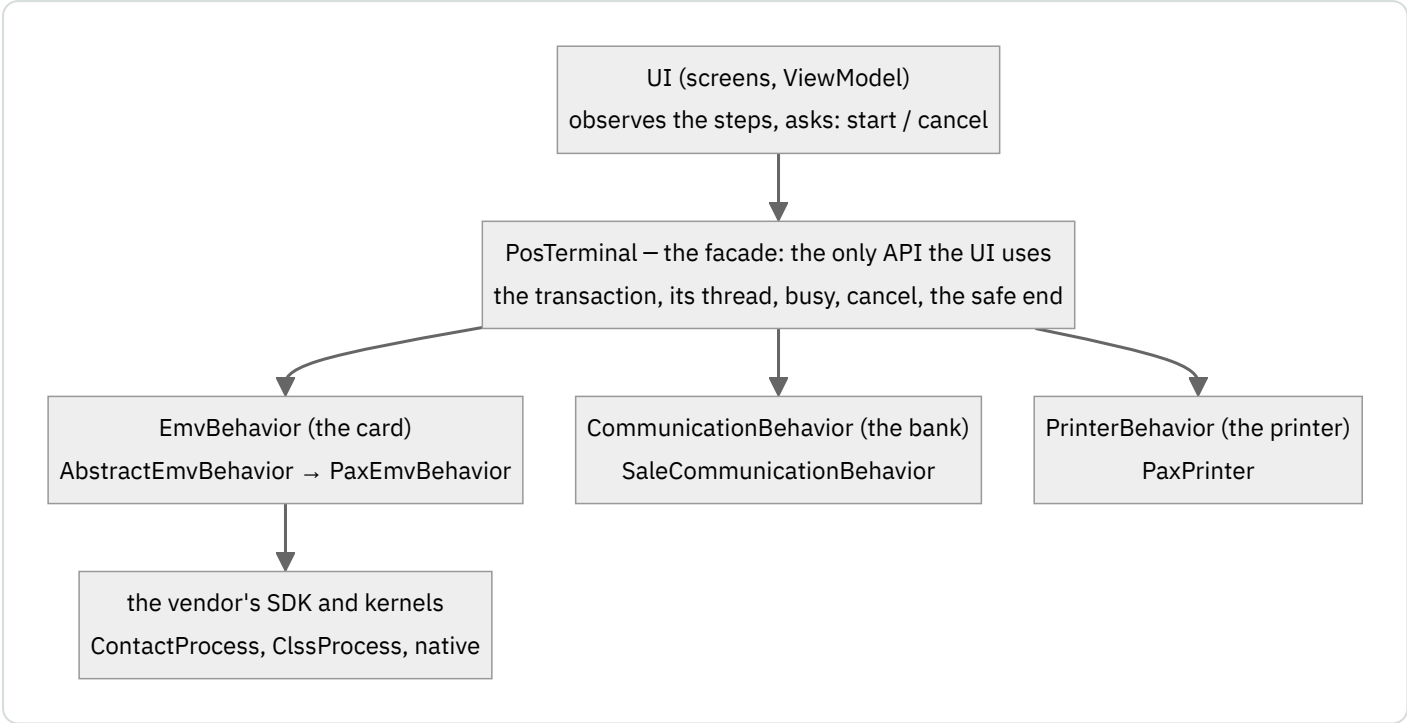
The first sale after a change



Param	Where it enters the kernel
Contact AIDs (the first one as the default)	`ContactProcess.preTransProcess` (`EMVAddApp` , `setEmvAndMCKParam`)
9F33 / 9F40 / 9F35 of the selected AID	`emvSetParam` → `resetParam` (inside `EMVReadAppData`)
CAPKs and revocations	Contact: `cardAuthentication` → `addCapk` . Contactless: the ODA stage.
Contactless AIDs and limits	`ClssEntryAddAid.addApp` + `Clss_PreTransProc_Entry`
TTQ, 9F33, floor, CVM limit, DF81xx	Each kernel's `setTransactionData` (granular), or inside the one call (atomic)
The TID in contactless field 55	`onReadCardOk` → `setTlv(9F1C)`

10. Design principles in the project

One line each: what it means, and where it shows in the code. Each one solves a real problem in payments.



Principle / pattern	In short	Where in the code
Layers + Facade	Each layer talks only to the one below; the UI sees one door.	`PosTerminal` is the UI's only door (`acceptCard` , `steps()` , `cancel()`). It is not an empty pass-through: it owns the sale.
Strategy / Polymorphism	One interface, several implementations, one chosen.	`EmvBehavior` (PAX / Ingenico / Fake), `CommunicationBehavior` , `PrinterBehavior`
Open / Closed	Add, don't edit.	A new vendor = a new flavor with its `TerminalFactory` and classes; the core does not change.
Template Method	The parent fixes the order; the child fills in the steps.	`AbstractEmvBehavior` : `run` , `goToStep` , the dispatch; the vendor fills the 16 `onXxx` hooks.
Observer	Whoever cares subscribes and gets every update.	`Transaction.steps()` and the screen; `emvSteps` and `EmvStepObserver` ; `TransactionStepObserver`
Callback	Give someone a number to call you back on.	`BankRequest` (§5), `CardSearchListener` , `IContactCallback` and `ask` (§4)
Adapter	A converter between two shapes that do not know each other.	`PaxHostAnswer` (`AuthResult` → PAX shape); `ContactProcess.EmvCallBackListener` (native → listener)
Factory / Builder	One place decides how a thing is made.	`TerminalFactory` (per flavor), `ClssKernelProcessFactory` (a kernel per scheme), `AuthResult.approved / declined / …` , `TransactionStepEvent.builder(...)`
Dependency Injection (by hand)	A class takes what it needs from outside instead of making it.	`PaxTerminal` → `super(behavior, communication, printer)` ; `attach(this::sendToBank)` for the one cycle
Dependency Inversion	The upper layer depends on an interface, not a concrete class.	The behavior knows `BankRequest` , not `PosTerminal` ; the core knows `CommunicationBehavior` , not Retrofit.
Single Responsibility	Each class has one job.	`PaxCardSearch` , `PaxPinEntry` , `PaxHostAnswer` , `PaxReaders` , `PaxCardData` , `PaxSetup` , `PaxKernelSession`
Single source of truth	Each piece of state has one owner and one copy.	Busy = `PosTerminal.current` ; the sale's state = its steps (`Transaction.decision()`); its steps = `Transaction.steps`
Immutability / Value Object	A thing never changes after it is made; a change gives a new copy.	`AuthResult` , `TransactionStepEvent` , `TransactionConfig.withXxx(...)`
Fail closed	When in doubt, refuse; never approve.	No answer = online failed; no PIN = `USER_CANCEL` ; a callback always answers (`setCallBackResult`)

11. Using us as an SDK (an app integrating us)

The idea: any app uses us for the card and EMV part. **The simplest contract:** the app gives us its bank and listens to the steps. `dispatchTransactionStep` (acknowledge and reverse) stays inside; the app never writes it.

Direction	What	How
App → us	**The bank**	Implements `CommunicationBehavior`: `connectToHost(request)` is required (send the sale, return an `AuthResult`); `acknowledge` / `reverse` are optional. The request arrives already named (`request.getAsyncRequestId()`).
App → us	Printing (optional)	`PrinterBehavior.print(lines)`
Us → app	**The steps**	`transaction.steps()`: `APPROVED` / `DECLINED` / `FAILED`, then the end `COMPLETED` / `ERROR` / `CANCELLED` (terminal free); the rest is progress. The stream completes after the end.
App → us	Start and cancel	`pos.acceptCard(type, amount)`, `transaction.cancel()`

```
// the integrator's bank — the only code they must write for the EMV part
class MyBank implements CommunicationBehavior {
    // required: send the sale, return the answer
    public Single<AuthResult> connectToHost(TransactionConfig request) { … }
    // optional
    public Completable acknowledge(TransactionConfig sent) { … }
    public Completable reverse(TransactionConfig sent) { … }
}

// their own screen (level B): start, observe, cancel
Transaction t = pos.acceptCard(TransactionType.SALE, amountMinor);
t.steps().subscribe(e -> {
    switch (e.getStep()) {
        case APPROVED:  /* approved + receipt fields */ break;
        case DECLINED:  /* declined */ break;
        case FAILED:    /* the error */ break;
        case COMPLETED:
        case ERROR:
        case CANCELLED: /* done, terminal free */ break;
        default:        /* progress: CARD_READ, ONLINE_* … */ break;
    }
});
cancelButton.setOnClickListener(v -> t.cancel());
```

- **Why our switch stays inside:** it protects the money. Acknowledge when the card approves, reverse when it declines, and reverse in `ENDING` after a cancel or a failure (a cancel can arrive while the bank is still answering). If every app wrote it, one would forget the reversal after a cancel and the customer would be charged. Inside, it is written once and covered by tests.
- **The request id is ours:** `sendToBank` names the request (`asyncRequestId`) before the app sees it; the acknowledgement and reversal carry the same id. The EMV never sees it.
- The switch in the app's code above is for its screen only; it does not drive the sale.

The simplest integration: our search-card screen (level A)

The app writes no screen: it opens ours with the amount and gets the result.

```
// once, in their Application
class App extends Application implements TerminalProvider {
    private PosTerminal pos;

    public void onCreate() {
        super.onCreate();
        pos = TerminalFactory.create(this, new MyBank(), new MyPrinter());
    }

    public PosTerminal terminal() {
        return pos;
    }
}

// to take a payment: one launch, one result
ActivityResultLauncher<PaymentRequest> pay =
        registerForActivityResult(new EasyLinkPayment(), outcome → {
            // APPROVED / DECLINED / FAILED / CANCELLED,
            // the message, the masked PAN
        });
pay.launch(new PaymentRequest(TransactionType.SALE, amountMinor));
```

The easiest steps to ship the SDK and the screen

#	Step	Why
1	`TerminalFactory.create(app, communication, printer)`: the bank and the printer come from outside. If the app sends none, the defaults stay (`SaleCommunicationBehavior`, `PaxPrinter`).	Today `PaxTerminal()` makes them itself, so an integrator cannot give us their bank. The constructor that takes them exists, but is `private`.
2	A new Android library module (e.g. `easylink-ui`) depending on `core` only: `SearchCardFragment`, `MainViewModel` (+ factory), `Event`, the three layouts and their strings.	The screen lives in `app` today. Resources get a prefix (`easylink_`) so they do not clash with the app's.
3	A `TerminalProvider` interface with `PosTerminal terminal()`, implemented by the app's `Application` and by `EmvEnhanceApp`.	The only link between the screen and the app. Today the screen casts to `EmvEnhanceApp`. Two implementations, so rule 5 allows it.
4	A `PaymentActivity` (or a contract like `EasyLinkPayment`) holding the fragment and returning the result through the Activity Result API.	Today Done only calls `finish()`; nobody gets the result.
5	Debug tools (the APDU trace on screen and its printing, the app-selection dialog even for one AID) in debug builds only.	They are for us, not for an integrator.
6	⚠ Before anyone else gets the SDK: the release gate (BACKLOG §5) — full PAN and field 55 on screen and receipt, `Level.BODY` logging, a real mToken in the code.	Shipped to someone else, that is leaked card data.

> The vendor (PAX) is still chosen by flavor and `TerminalFactory`, so an integrator takes `core`, `vendor-pax` (and its libraries) and `easylink-ui`. This is a direction only; no code yet.

Note: the difference with `paxEmvBehaviorSplit`

The older branch `paxEmvBehaviorSplit` (and `-tests`) works the same way, with older names and details: `authorizeOnline` / `Authorizer` / `PosTerminal.authorize` / `CommunicationBehavior.authorize`; `START_ONLINE_PROCESS`; `tx.lastAuth` in each vendor; `CLOSED` instead of `CLOSING_COMPLETED`; no `ENDING` (the end through `onEnd` and a loop); the switch called from several places; the network as an Rx chain (`flatMap`) with no `callTimeout`. The full list is in `doc/REFERENCE.md`.